Testing Pauli Propagation on Variational Benchmarks for Quantum Many-Body Problems

Master's Thesis

Julia Gerecke

January 23, 2026

Supervisor: Zoë Holmes
Laboratory of Quantum Information and Computation
École Polytechnique Fédérale de Lausanne

Abstract

Quantum simulations are promising for studying quantum-chemical problems, such as the preparation of molecular and lattice Hamiltonians' ground states, with the ultimate goal of advancing materials discovery. For efficient use of quantum hardware, we require a classical ground-state preparation framework that can serve as an input and benchmark for quantum hardware simulations. We implement and benchmark variational ground-state preparation using the classical simulation algorithm Pauli Propagation. Pauli Propagation computes expectation values in the Heisenberg picture by back-propagating observables through a circuit. Even though the algorithm relies on truncations to remain computationally feasible and scalable, it has proven to be accurate in many applications. The ansatz we use to approximate the ground state is ADAPT-VQE, a hybrid quantum-classical algorithm which we have adapted to work fully classically. To enable scaling to larger system sizes, we employ an operator-pool tiling strategy. This takes operators relevant for expressing the ground state of small systems and extends them to match the larger system size. We focus on spin Hamiltonians in 1D and 2D and validate approximate ground states by re-evaluating them at stricter truncation settings and by checking symmetry-based observables. We benchmark the accuracy of our results against exact solutions where available, as well as against state-of-the-art numerical methods such as Density Matrix Renormalization Group and Neural Quantum States. We further examine the scalability of our approach by considering time and memory behaviour in interplay with the accuracy of specific implementation decisions, such as the optimization procedure, the operator tile size, as well as threading options on high-performance computers.

Contents

1 Introduction **4**

2 Preliminaries **5**
2.1 Pauli Propagation 5
2.2 ADAPT-VQE 7
2.3 Physical Systems of Interest 11

3 Results **13**
3.1 Algorithmic and Software Development 13
3.2 Simulations of Physical Models 13

4 Conclusion **19**

Bibliography **22**

5 Appendix **23**
5.1 Threading of the Propagation 23
5.2 Single Run Timing and Memory Analysis 26
5.3 Loop Implementation 26
5.4 Tiling Comparison 28
5.5 Spin and Fermionic Hamiltonians 29
5.6 Observables for Symmetry Verification 30
5.7 Optimization Algorithm 32
5.8 Further Implemented Ground State Preparation Algorithms 35
5.9 Comparison to Previous Propagation-Based Results 35
5.10 V-score Computation 36

1 Introduction

Quantum computing aims to address classically intractable problems by efficiently simulating quantum systems and preparing highly correlated quantum states. A prominent application domain is quantum chemistry, where quantum algorithms for Hamiltonian simulation and energy estimation offer favorable asymptotic scaling with system size compared to exact classical methods [1]. While these approaches suggest a path toward treating chemically realistic models of increasing complexity, significant progress is still required on both the quantum hardware and quantum algorithms side to reach a practical advantage.

A fundamental problem in quantum chemistry is to determine the ground state of a physical Hamiltonian. This includes molecular Hamiltonians, models of strongly correlated materials (e.g., Hubbard-type models), and effective spin Hamiltonians. Preparing the ground state of a quantum system is crucial for understanding its physical properties, as it provides the reference point for excited-state calculations and dynamical response [2].

The use of classical simulation algorithms to benchmark and improve quantum ground-state preparation methods is an active area of research. Purely classical simulations can provide reference data for quantum hardware computations, which are currently limited by noise and qubit count in the Noisy Intermediate-Scale Quantum (NISQ) era [3]. With current classical methods, ground states for medium-sized systems (often up to $\mathcal{O}(10^2)$ spins/qubits, depending on the model, dimension, and topology) can be simulated, surpassing the capabilities of current quantum hardware.

Beyond the direct utility of ground states for predicting stability and low-energy phases, ground-state preparation is also the algorithmic "front end" for many quantum simulation workflows: methods such as phase estimation, imaginary-time-inspired procedures, or variational ansätze, can all benefit substantially from being initialized in a state with non-negligible overlap with the true ground state [4, 5]. Thus, one can view classical methods as subroutines for efficiently preparing an approximation to the ground state for subsequent quantum hardware computations. In practice, the availability of accurate, scalable approximate ground states is a central bottleneck for both classical computations and near-term quantum devices with limited circuit depth and measurement budgets.

A recurring difficulty in this benchmarking loop is that the exact ground-state energy is typically unknown at the system sizes and geometries most relevant for strongly correlated physics. Consequently, it is valuable to have an internal, method-agnostic indicator of variational quality that does not rely on cross-validation against exact diagonalization or other methods. A natural diagnostic is the energy variance, which vanishes on eigenstates and can therefore serve as a quantitative measure of distance to an eigenstate. Recent work proposes combining the variational energy and its variance into a dimensionless accuracy metric (called V-score) [6]. The V-score identifies regimes where state-of-the-art classical approaches show limited accuracy and where quantum algorithms (or improved classical approaches) are most plausibly impactful.

There have been many recent developments in the design of variational methods for ground-state preparation. Prominent examples include Tensor Networks [7–11] and Neural Quantum States (NQS) [6, 12]. A recent development is the use of operator propagation methods, such as Pauli Propagation (PP) [13–17], or Majorana Propagation (MP) [18, 19]. These methods were initially developed for simulating dynamics, but have recently been adapted for ground-state preparation, for example, using ADAPT-VQE [18, 20], the Hamiltonian Variational Ansatz (HVA) [21], or the variational Double-Bracket Flow (vDBF) [22].

In this thesis, we implement and benchmark variational ground state preparation using the classical simulation algorithm Pauli propagation. We focused on simulating spin Hamiltonians. To enable

scaling to larger system sizes, we employ an operator-pool tiling strategy [23]. We validate the resulting approximate ground states using re-evaluation at stricter truncation settings and by checking symmetry-based observables. We quantify accuracy using available reference energies and compute V-scores for selected systems. Finally, we analyze practical performance aspects that indicate further scalability, including timing, peak memory usage, and multi-threading behavior.

Pauli propagation is a classical algorithm most commonly used to compute expectation values of observables. It operates in the Heisenberg picture, where observables are back-propagated through the circuit, rather than the Schrödinger picture, where quantum states are evolved forward. The efficiency of the method relies on the fact that physically relevant states and observables can often be represented as a sum of a comparatively small number of Pauli strings. Under generic non-Clifford rotations, the number of terms in the propagated operator can grow rapidly due to branching. Practical implementations, therefore, rely on controlled approximations (e.g., coefficient or weight truncation) which aim to dicard less relevant contributions to the final expectation value [13, 16]. From the perspective of variational ground-state preparation, this representation is practical because many cost functions and diagnostics can be expressed directly in terms of expectation values of Pauli-sum Hamiltonians and related observables. Notably, it is not a priori clear that Pauli propagation will remain sufficiently accurate in this role under the truncations required for scalability. Nevertheless, we find in practice that Pauli propagation provides reliable estimates across the benchmarked instances.

2 Preliminaries

2.1 Pauli Propagation

Pauli propagation is a classical simulation method for quantum circuits [13–17]. The term "Pauli propagation" was first introduced in 2019 [13]. The purpose of this classical algorithm is to efficiently calculate expectation values of observables. Unitary or isometric dynamics are described in the Schrödinger picture by the quantum channel

$$\mathcal{E}_{\underline{\theta}}(\rho) \;=\; U(\underline{\theta})\,\rho\,U^{\dagger}(\underline{\theta}), \tag{2.1.1}$$

where $\underline{\theta}$ are the circuit parameters and $U(\underline{\theta})$ is the (iso-)unitary implemented by the circuit. In the following, we evaluate the channel on an initial density operator ρ_0. The expectation value of an observable $\hat{O}$ is given by

$$f(\underline{\theta}) = \mathrm{Tr}\big[\hat{O}\,\mathcal{E}_{\underline{\theta}}(\rho_0)\big] = \mathrm{Tr}\big[\hat{O}\,U(\underline{\theta})\rho_0 U^{\dagger}(\underline{\theta})\big]. \tag{2.1.2}$$

To switch to the Heisenberg picture, we introduce the adjoint (dual) channel $\mathcal{E}^{\dagger}_{\underline{\theta}}$ defined by

$$\mathrm{Tr}\big[\hat{O}\,\mathcal{E}_{\underline{\theta}}(\rho)\big] \;=\; \mathrm{Tr}\big[\mathcal{E}^{\dagger}_{\underline{\theta}}(\hat{O})\,\rho\big] \qquad \forall\,\rho,\hat{O}. \tag{2.1.3}$$

For unitary or isometric $U(\underline{\theta})$ this adjoint map is simply conjugation,

$$\mathcal{E}^{\dagger}_{\underline{\theta}}(\hat{O}) \;=\; U^{\dagger}(\underline{\theta})\,\hat{O}\,U(\underline{\theta}), \tag{2.1.4}$$

and therefore

$$f(\underline{\theta}) \;=\; \mathrm{Tr}\big[\mathcal{E}^{\dagger}_{\underline{\theta}}(\hat{O})\,\rho_0\big] \;=\; \mathrm{Tr}\big[\rho_0\,U^{\dagger}(\underline{\theta})\hat{O}U(\underline{\theta})\big]. \tag{2.1.5}$$

In a circuit composed of multiple gates, $\mathcal{E}^{\dagger}_{\underline{\theta}}$ corresponds to propagating the observable "backwards" through the circuit, which is why we refer to backpropagating observables in the Heisenberg picture. Note that Equation 2.1.2 and Equation 2.1.5 are equal by the cyclity of trace.

Pauli Transfer Matrix formalism. We can express any unitary quantum circuit in the general form

$$U(\underline{\theta}) = \prod_{i=1}^{m} C_i \, e^{-i\frac{\theta_i}{2}P_i}, \tag{2.1.6}$$

where C_i are Clifford gates and $e^{-i\frac{\theta_i}{2}P_i}$ are Pauli rotations generated by Pauli strings P_i. Pauli propagation operates in the Pauli Transfer Matrix (PTM) formalism, which represents the Heisenberg evolution (2.1.4) as a linear map acting on Pauli coefficients. The PTM formalism works in the n-qubit Pauli basis $\{P_i\}_{i=1}^{4^n}$, so an observable can be written as

$$\hat{O} = \sum_{i=1}^{N_O} c_i P_i, \tag{2.1.7}$$

where we sum over the N_O non-zero coefficients. Since the observables of interest are often local and thus sparse in this basis, propagating $\hat{O}$ can be significantly cheaper than propagating quantum states, which are typically dense in the Pauli basis.
Let $\mathcal{U}_{\underline{\theta}} := \mathcal{E}_{\underline{\theta}}^{\dagger}$ denote the conjugation channel on observables, $\mathcal{U}_{\underline{\theta}}(\cdot) = U^{\dagger}(\underline{\theta})(\cdot)U(\underline{\theta})$. The (unnormalized) PTM of $U(\underline{\theta})$ is the matrix U_{PTM} with entries

$$\big(U_{\mathrm{PTM}}\big)_{ij} \;=\; \mathrm{Tr}\big(P_i\,\mathcal{U}_{\underline{\theta}}(P_j)\big) \;=\; \mathrm{Tr}\big(P_i\,U^{\dagger}(\underline{\theta})P_jU(\underline{\theta})\big) \;=\; \mathrm{Tr}\big(U(\underline{\theta})P_iU^{\dagger}(\underline{\theta})\,P_j\big), \tag{2.1.8}$$

where the last equality follows from the cyclicity of the trace. The conjugate channel applied to P_j results in a linear action in the PTM formalism by expansion in the Pauli basis,

$$\mathcal{U}_{\underline{\theta}}(P_j) \;=\; U^{\dagger}(\underline{\theta})P_jU(\underline{\theta}) \;=\; \sum_i a_{ij}P_i, \qquad a_{ij} = \frac{1}{2^n}\,\mathrm{Tr}\big(P_i\,\mathcal{U}_{\underline{\theta}}(P_j)\big) = \frac{1}{2^n}(U_{\mathrm{PTM}})_{ij}, \tag{2.1.9}$$

where we used $\mathrm{Tr}(P_iP_j) = 2^n\delta_{ij}$ in the Pauli basis.
Therefore, for $\hat{O} = \sum_i c_iP_i$, the coefficient vector update to describe the conjugate channel is

$$\underline{c}' \;=\; \frac{1}{2^n}\,U_{\mathrm{PTM}}\,\underline{c}. \tag{2.1.10}$$

Pauli propagation exploits this Heisenberg-picture viewpoint by propagating sparse Pauli expansions of observables through the circuit, which is advantageous whenever the relevant operators remain comparatively sparse under conjugation.

Branching under Pauli rotations. When applying the rotation gates of the circuit to a Pauli string in the observable, there are two possibilities:

$$e^{i\frac{\theta_i}{2}P_i}\,P_j\,e^{-i\frac{\theta_i}{2}P_i} = \begin{cases} P_j, & \text{if } [P_i,P_j]=0, \\ \cos(\theta_i)\,P_j + \sin(\theta_i)\,P_{j'}, & \text{if } \{P_i,P_j\}=0, \end{cases} \tag{2.1.11}$$

where in the anticommuting case, the new Pauli string is $P_{j'} := \frac{i}{2}[P_i,P_j]$. In the first case, the generator of the Pauli rotation P_i and the Pauli string P_j commute, leaving the Pauli string invariant under application of the gate. In the second case, they anticommute, and conjugation maps a single input Pauli string to two Pauli strings. The splitting of one term into multiple terms is referred to as branching (or path branching) and is the source of growth in the number of terms in the Pauli-sum representation. A Pauli rotation with $\{P_i,P_j\}=0$ therefore generally increases the number of strings, also referred to as the "weight" of the Pauli sum. It should not be confused with the "weight" of a single Pauli string, which is the number of non-identity Paulis in a string. In practice, some generated strings may coincide with strings already present in the sum. In that case, the corresponding coefficients are combined, which is known as path merging. Without an effective truncation strategy, repeated

branching causes the Pauli expansion to grow exponentially in the number of terms. Truncation is therefore essential to keep simulations of deep circuits computationally feasible.

Non-branching Clifford propagation. We briefly note the simple propagation rule that applies when the unitary evolution consists solely of Clifford gates. By the definition of a Clifford gate, conjugation maps any Pauli string to a single Pauli string up to a sign

$$C_i^\dagger P_j C_i \;=\; \pm P_{j'}, \tag{2.1.12}$$

where the output Pauli string $P_{j'}$ and the sign are uniquely determined by the Clifford gate C_i and input Pauli string P_j. Since the number of terms in a Pauli expansion is preserved under Clifford conjugation, Clifford updates are "non-branching". Each term is mapped one-to-one rather than generating additional terms. This mapping can be implemented efficiently via bitwise operations: Pauli strings are represented as bit strings, and Clifford gates can be implemented using bitwise logic. Although this operation does not immediately increase the number of terms, the resulting Pauli string may have a higher probability of branching. A typical example of this is the CNOT gate. In the remainder of this work, we restrict ourselves to circuits generated by Pauli rotations and therefore do not explicitly rely on separate Clifford-specific implementation details.

Truncation strategies. Different truncation schemes exist to make the computation tractable. The most intuitive truncation strategy is to discard all terms with absolute coefficients below a minimum value since these Pauli strings do not significantly contribute to the final expectation value. We refer to this as setting the minimum absolute coefficient (mac) in our code. Another possible choice is weight truncation, where "weight" denotes the number of non-identity operators in a given Pauli string (see Ref. [24] for theoretical guarantees for unstructured circuits). Say we are interested in the overlap of the observable with the all-zero state, then the expectation value

$$\langle 0^{\otimes n}| \, P_i \, |0^{\otimes n}\rangle = 0, \tag{2.1.13}$$

if any of the Pauli matrices in the Pauli string are X or Y. Thus, a high-weight term is more likely to have an expectation value of zero. This is only an intuition for structured circuits, while theoretical guarantees for weight truncation hold only for unstructured or random circuits. In our work, we apply weight truncation primarily as an intuition: despite the structured nature of the ansätze considered here, we observe empirically that setting a maximum weight (mw) can yield accurate expectation values.

A key practical instantiation of the above framework is `PauliPropagation.jl` [16], an open-source Julia package which makes Pauli propagation usable as a high-performance simulator for large circuit instances. The implementation is scalable primarily through leveraging low-level data representations. Pauli strings are encoded as unsigned integers, allowing the Pauli update rule (Equation 2.1.11) to be implemented with constant-time bitwise operations. The advantage of such low-level implementations is that they map directly to CPU instructions and can be processed in the arithmetic logic unit, making them significantly faster than floating-point arithmetic operations. `Julia`'s multiple dispatch keeps the interface extensible, allowing users to implement custom operations.

2.2 ADAPT-VQE

The Adaptive, Problem-Tailored Variational Quantum Eigensolver (ADAPT-VQE, or ADAPT) [20, 25] is a variational ground-state preparation algorithm that constructs an ansatz by selecting one operator from a pool at each iteration, rather than optimizing a fixed, full-depth circuit in one optimization function call. By growing the circuit layer-by-layer, the method repeatedly solves optimization problems with warm-started parameters, which is often empirically more robust than simultaneously optimizing all randomly initialized parameters (e.g., it is less susceptible to barren plateaus). The first instances of ADAPT used an excitation-based pool and were extended to qubit-ADAPT [26], which draws from a pool of Pauli strings instead. Therefore, we can implement qubit-ADAPT directly

within the Pauli propagation framework. At each ADAPT iteration, the current ansatz is optimized by the Variational Quantum Eigensolver (VQE) [27]. This hybrid quantum-classical algorithm aims to minimize the energy $E(\underline{\theta}) = \mathrm{Tr}[\hat{H}\ U(\underline{\theta})\rho_0 U^\dagger(\underline{\theta})]$ using quantum measurements of $\langle \hat{H} \rangle$ and a classical optimizer over $\underline{\theta}$. The classically optimized parameters are fed back into the parametrized circuit on the quantum hardware for subsequent measurement. The process is repeated until convergence. We run an entirely classical version of VQE by simply replacing all quantum computations with PP.

Summary of the ADAPT-VQE algorithm. The following is a compact summary of the algorithm; the subpoints are explained in detail below.

(1) Initialisation: Initialize a Hamiltonian $\hat{H}$, an initial state $|\psi_0\rangle$, an operator pool $\mathcal{P}$, and convergence criteria.

(2) Operator selection via pool gradients: For each candidate operator $P \in \mathcal{P}$, append a trial layer (where P generates the Pauli rotation) and calculate the energy gradient evaluated at zero amplitude of the new layer. Choose the trial operator with the largest absolute gradient value, where ties are broken randomly.

(3) Gradient convergence: Check if the gradient convergence criteria is fulfilled. For example, check whether the maximum absolute gradient of the chosen operator is below a specific threshold or whether it has been stagnating in previous layers. If yes, terminate; if not, continue.

(4) Appending a new layer and VQE: Append the Pauli rotation generated by the selected operator. Perform a VQE optimization over all variational parameters (warm-starting from the previous optimum), then return to **(2)**.

Ansatz structure. In qubit-ADAPT, we build the variational ansatz iteratively from an operator pool of Pauli strings. At iteration k, we select an operator generator iP_k, which is anti-Hermitian. Given the current variational state $|\psi_{k-1}(\theta_1, \ldots, \theta_{k-1})\rangle$, the k-th state is obtained by appending one additional rotation gate generated by the newly selected iP_k,

$$|\psi_k(\theta_1, \ldots, \theta_k)\rangle = e^{i\theta_k P_k}\, |\psi_{k-1}(\theta_1, \ldots, \theta_{k-1})\rangle\,. \tag{2.2.1}$$

Using the conventional rotation-gate parameterization

$$e^{i\theta_k P_k} \;=\; e^{-i\frac{\Theta_k}{2} P_k}, \qquad \Theta_k = -2\theta_k. \tag{2.2.2}$$

Since we are working in the Pauli propagation setting, the ansatz consists of Pauli rotations. In general, the ADAPT ansatz requires anti-Hermitian operator generators ($\hat{\tau}_k = -\hat{\tau}_k^\dagger$), and can be applied to different types of systems. For fermionic simulations, one can either work with fermion to qubit mappings or use Majorana monomials as generators [18]. In the molecular ADAPT variation, the generators are excitation-like operators [20].

Tiles approach. For Pauli strings on n qubits, the full operator pool size scales as $4^n - 1$ (excluding the global identity). This operator pool size is only feasible for small n, motivating the use of structured pool constructions for larger systems. To avoid the exponential number of operators in the pool, we can adopt a tiling approach. It takes an operator pool for a small sub-system (for example, the full operator pool of dimension $4^{n_{\mathrm{tile}}} - 1$) and tiles this across the scaled system size. Tiling means that the local operators are padded by identities on all non-acting qubits. In one-dimensional chains, this tiling reduces to shifting short Pauli strings along the chain, subject to the chosen boundary conditions: for open boundaries, operators shift until they reach the end of the chain, whereas for periodic boundaries, shifting stops when the tile would overlap with the first operator.
Instead of using the full operator pool for the tile-sized system, we further reduce the pool size, as introduced in [23]. Concretely, we first run multiple instances of ADAPT-VQE on a smaller tile size with $\mathrm{n}_{\mathrm{tile}}$ qubits using the full operator pool and iterate until we meet the usual convergence criterion.

We then collect the set of unique operators selected during these small-system runs and promote them to an effective pool for the target system by translating each chosen operator across the larger lattice and again padding with identities, as illustrated in Figure 2.1. Since the Hamiltonians considered are local (up to next-nearest-neighbor terms in the J_1-J_2 case), operators identified as relevant on a small local region are expected to remain relevant when embedded into larger instances. This assumes that the physics of the small system translates well to the scaled system. The existence of compact exact pools, i.e., minimal-sized pools that contain an exact representation of the ground state, is known to have a size of $2n - 2$ [28]. The greedy, iterative nature of ADAPT, however, does not guarantee that the ansatz traverses the pool in an ordering that yields the exact ground state, and we further do not have access to the minimal pool in general.
The tiling pool is a compromise: On one hand, it scales linearly with system size while retaining operators that are empirically important on representative local instances. On the other hand, the heuristic does not consider symmetry properties of the Hamiltonian, which are an alternative or additional way of restricting the pool, as explained in paragraph 5.6. The tiling approach can yield an ansatz that violates key symmetries of the Hamiltonian. Further, we might require multiple re-runs of the same instance in order to traverse through the pool in an optimal manner, since for degenerate operator gradients, the appended operator is chosen at random.

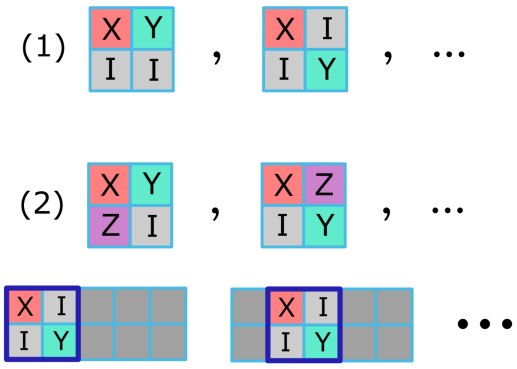


Figure 2.1: Illustration of the tiles approach for two dimensions, taken from [23]. **(1)** and **(2)** are the collections of operators chosen by the two instances of small ADAPT run, for example, X_1Y_4 (upper right). To generate the scaled pool, for a target lattice of 2×5 and open boundary conditions, each tile occupies four positions (padded with identities on the dark grey fields).

Operator selection via pool gradients. Given a pool $\mathcal{P}$, ADAPT requires a criterion to select an operator at each iteration. In standard (unitary) ADAPT-VQE, the selection metric is the magnitude of the energy gradient with respect to the newly appended parameter (Step **(2)**). For unitary evolutions, we can express this derivative in terms of the expectation value of a commutator, and efficiently calculate it approximately in PP.
The ansatz after appending the k-th generating Pauli string is

$$|\psi_k(\theta_1, \ldots, \theta_k)\rangle = e^{i\theta_k P_k} |\psi_{k-1}(\theta_1, \ldots, \theta_{k-1})\rangle . \tag{2.2.3}$$

We can express the energy at iteration k as

$$E(\theta_k) = \langle\psi_k| \hat{H} |\psi_k\rangle = \langle\psi_{k-1}| e^{-i\theta_k P_k} \hat{H} e^{i\theta_k P_k} |\psi_{k-1}\rangle . \tag{2.2.4}$$

To obtain the gradient of the energy, we differentiate with respect to θ_k

$$\frac{\partial E}{\partial \theta_k} = \left\langle \psi_{k-1} \middle| \frac{\partial}{\partial \theta_k}\left(e^{-i\theta_k P_k}\hat{H}e^{i\theta_k P_k}\right) \middle| \psi_{k-1} \right\rangle \tag{2.2.5}$$

$$= \langle \psi_{k-1}| e^{-i\theta_k P_k}\left[\hat{H}, iP_k\right] e^{i\theta_k P_k} |\psi_{k-1}\rangle \tag{2.2.6}$$

$$= \langle \psi_k| \left[\hat{H}, iP_k\right] |\psi_k\rangle, \tag{2.2.7}$$

where we used the chain rule. We evaluate the expression at $\theta_k = 0$, so at the previously optimised circuit structure. In the second line we used that $[iP_k, e^{\pm i\theta_k P_k}] = 0$, since iP_k commutes with any analytic function of itself.

Approximation of the gradient under truncated Pauli propagation. The commutator from Equation 2.2.5 holds only for unitary evolution, i.e., when the Heisenberg-evolved operator after one layer is $\hat{H}(\theta_k) = e^{-i\theta_k P_k}\hat{H}e^{i\theta_k P_k}$. Since Pauli propagation relies on truncation, however, the exact operator is replaced by an approximate representation

$$\widetilde{H}(\theta_k) \equiv \mathcal{T}\left(e^{-i\theta_k P_k}\hat{H}e^{i\theta_k P_k}\right), \tag{2.2.8}$$

where $\mathcal{T}$ discards Pauli terms. While merging is an exact operation, discarding Pauli terms is not. Since $\mathcal{T}$ is not (in general) an algebra homomorphism, it does not preserve products and commutators, so

$$\frac{\partial}{\partial \theta_k}\widetilde{H}(\theta_k) \;=\; \frac{\partial}{\partial \theta_k}\mathcal{T}\Big(\hat{H}(\theta_k)\Big) \;\neq\; \mathcal{T}\left(\frac{\partial \hat{H}(\theta_k)}{\partial \theta_k}\right) \;=\; \mathcal{T}\Big([\hat{H}(\theta_k), iP_k]\Big) \;\neq\; [\widetilde{H}(\theta_k), iP_k]. \tag{2.2.9}$$

Consequently, evaluating gradients via the commutator identity becomes an approximation whose accuracy depends on the truncation scheme and threshold. Alternatively, one can compute energy gradients by differentiating through the truncated propagation, for example, using automatic differentiation.

Truncated Pauli propagation and the variational principle. The variational principle states that any normalized variational state $|\psi(\underline{\theta})\rangle$ energy is above the true ground state energy E_0

$$E(\underline{\theta}) = \langle \psi(\underline{\theta})| \hat{H} |\psi(\underline{\theta})\rangle \geqslant E_0, \tag{2.2.10}$$

This lower bound follows from the spectral decomposition of $\hat{H}$ and the fact that $E(\underline{\theta})$ is a convex combination of eigenvalues when expressed in the eigenbasis of $\hat{H}$. Variational methods such as the VQE, variational Monte Carlo, and NQS may use an ansatz family that is not expressive enough to represent the exact ground state, and are approximate in that sense. Nevertheless, they remain variational because they evaluate the energy as the expectation value of $\hat{H}$ in a normalized state (i.e., as a self-consistent Rayleigh quotient). In Pauli propagation, by contrast, truncation is applied during the Heisenberg back-propagation. The propagated operator does not coincide with the exact conjugate operator, and the overall truncated update does not correspond to the adjoint of any physical channel. Consequently, the resulting estimate $\widetilde{E}(\underline{\theta})$ is not guaranteed to equal $\mathrm{Tr}[\hat{H}\rho(\underline{\theta})]$ for any normalized state $\rho(\underline{\theta})$, so the variational principle does not hold.
This is not inherently an issue. The goal of using truncations is to have a scalable, classical approach which guides the optimization toward parameters $\underline{\theta}$ that perform well for the untruncated circuit $U(\underline{\theta})$, i.e., the circuit we ultimately aim to run on quantum hardware. While truncation can produce a non-variational objective (for example, when $\widetilde{E}(\underline{\theta})$ falls below E_0 or exhibits other non-physical artifacts), it substantially reduces the classical cost compared to PP without truncation (exact propagation) and other high-accuracy simulators. In our numerical experiments, the parameters obtained from truncated PP represent the ground state well. We verify this by re-evaluating the learned circuits at stricter truncation settings or under exact simulation, where, due to unitary evolution, all energies satisfy the variational bound. Further, comparing the truncated-run values against the re-evaluation provides a

diagnostic of whether the truncated propagation preserved the physically relevant operator content. If the truncated run and the exact re-evaluation are close in expectation values, the truncated run is a good representation of the full propagation. For too aggressive truncations, the energy curves deviate with increasing circuit depth. In general, we always need to perform less truncated re-evaluations to gain confidence in the calculated expectation values.
We could alternatively design a variational approximation using PP by explicitly enforcing normalization within the truncated representation (for example, by optimizing a self-consistent Rayleigh-quotient objective). However, this would amount to optimizing the ground state of the truncated surrogate model defined by the truncation rule, rather than optimizing the energy of the intended physical circuit, which is not our goal.

Gradient convergence. In unitary applications of ADAPT, for example, on a quantum device, a vanishing pool-gradient norm is the only termination condition. If all pool gradients are below a threshold, then no first-order improvement direction remains in the span of the operator pool $\mathcal{P}$ (Step **(3)**). In large-scale truncated evolutions, we observe, however, that the operator gradients do not approach zero cleanly. Instead, we observe a plateau in the gradient caused by truncation error. Therefore, we expand the stopping criteria beyond the standard pool-gradient threshold by combining it with a plateau criterion (no statistically significant decrease in the maximum pool gradient over several iterations). Additionally, the optimization stops after a maximum number of ADAPT iterations.

Appending a new layer and VQE. If the algorithm has not converged, we append the operator with the largest absolute pool-gradient (Step **(4)**). The new parameter θ_k is initialized at random. Due to the selection rule, the selected operator has a non-zero first-order effect at the current state. Nevertheless, we avoid initializing exactly at $\theta_k = 0$ (which yields the previous state) since it can be numerically inconvenient, as line-search or curvature estimates are poorly conditioned when introducing the new direction. In practice, we either sample θ_k from a small interval around zero or from the full angle range. After appending the new operator, we run the PP version of VQE and re-optimize all parameters using a classical optimizer. We warm-start the optimization by initializing $(\theta_1, \ldots, \theta_{k-1})$ at the previously optimized values and only randomizing the newly introduced θ_k. In our implementation, we employ L-BFGS as the inner-loop optimizer.

2.3 Physical Systems of Interest

We study several canonical quantum many-body models as benchmarks for ground-state preparation via ADAPT-VQE with Pauli propagation. These models span a range of physical settings, including spin systems and fermionic lattice models, providing a diverse testbed for the proposed methods. We use the conventions and model parameters chosen in [6].

Transverse-field Ising model (TFIM). We consider the spin-$\frac{1}{2}$ TFIM,

$$\hat{H} = J \sum_{\langle i,j \rangle} \hat{\sigma}_i^z \hat{\sigma}_j^z + h \sum_i \hat{\sigma}_i^x, \tag{2.3.1}$$

with nearest-neighbor couplings $\langle i, j \rangle$ and Pauli operators $\hat{\sigma}^a$, setting $J = h = 1.0$.
We focus on the critical regime ($h/J \simeq 1$) to probe genuinely nontrivial correlations. Deep in the antiferromagnetic phase, the ground state is essentially the Néel product state, making ground-state preparation and validation simple. The plus state is a suitable choice for the initial state of the TFIM, since this is the exact solution in the limit of a dominant external field and therefore has a non-zero overlap with the target state.

Heisenberg model. We study the SU(2)-symmetric Heisenberg Hamiltonian,

$$\hat{H} = J \sum_{\langle i,j \rangle} \sum_{a \in x,y,z} \hat{\sigma}_i^a \hat{\sigma}_j^a, \tag{2.3.2}$$

with $J = 1$. This model is interesting to study as it is an effective description of exchange physics in Mott insulators. It supports nontrivial many-body behavior (e.g., criticality in one dimension and strong quantum fluctuations in higher dimensions), making it a standard benchmark Hamiltonian. We choose the initial state to be the Néel state, which is the classical limit of the ground state.

J_1-J_2 model. To include frustration from competing interactions, we consider the J_1-J_2 model

$$\hat{H} = J_1 \sum_{\langle i,j \rangle} \sum_a \hat{\sigma}_i^a \hat{\sigma}_j^a + J_2 \sum_{\langle\langle i,j \rangle\rangle} \sum_a \hat{\sigma}_i^a \hat{\sigma}_j^a, \tag{2.3.3}$$

where $\langle\langle i,j \rangle\rangle$ denotes next-nearest neighbors. We set $J_1 = 1$ and $J_2 = \frac{1}{2}$. On a square lattice, we take the diagonal lattice points to be the next-nearest neighbors. The next-nearest-neighbor term introduces frustration and qualitatively new correlations. The model is exactly solvable at the Majumdar–Ghosh point $J_2/J_1 = 1/2$ in 1D. As an initial state, we also use the Néel state since it has non-zero overlap with the ground state.

t-V model (spinless fermions). As a minimal interacting fermion problem, we use

$$\hat{H} = -t \sum_{\langle i,j \rangle} \left(\hat{c}_i^\dagger \hat{c}_j + \hat{c}_j^\dagger \hat{c}_i \right) + V \sum_{\langle i,j \rangle} \hat{n}_i \hat{n}_j, \tag{2.3.4}$$

where $\hat{c}_i^\dagger$ and $\hat{c}_i$ are fermionic creation and annihilation operators for a spinless fermion on lattice site i, obeying the canonical anticommutation relations $\{\hat{c}_i, \hat{c}_j^\dagger\} = \delta_{ij}$ and $\{\hat{c}_i, \hat{c}_j\} = 0$. The first term describes nearest-neighbor hopping with amplitude t, and the second term is a nearest-neighbor density–density interaction of strength V. We set $t = 1$ to fix the energy scale, define the occupation (number) operator as $\hat{n}_i = \hat{c}_i^\dagger \hat{c}_i$, and work at fixed particle number.
This model is a first step toward simulating realistic fermionic Hamiltonians relevant to materials and molecular electronic structure, and we represent it on qubits via a Jordan-Wigner mapping, which maps fermionic operators to Pauli strings, thereby enabling digital simulation. Due to the Jordan-Wigner mapping, we can also use the Néel state as an initial state, which corresponds to a Fock state at half-filling via the fermion-to-qubit mapping.

Hubbard model. We also consider the spinful Hubbard Hamiltonian,

$$\hat{H} = -t \sum_{\langle i,j \rangle, \sigma} \left(\hat{c}_{i\sigma}^\dagger \hat{c}_{j\sigma} + \hat{c}_{j\sigma}^\dagger \hat{c}_{i\sigma} \right) + U \sum_i \hat{n}_{i\uparrow} \hat{n}_{i\downarrow}, \tag{2.3.5}$$

with $t = 1$, on-site interaction strength U, and fixed total particle numbers $N_\uparrow = N_\downarrow$. The Hubbard model is the lattice model of strong electronic correlations and a common test Hamiltonian for quantum simulation. As in the t-V case, we employ the Jordan–Wigner transformation to encode fermionic anticommutation relations into qubit-native Pauli operator strings. This fermion-to-qubit mapping introduces nonlocal Pauli strings, making it more challenging for the tiling approach, which is local in the Pauli basis.

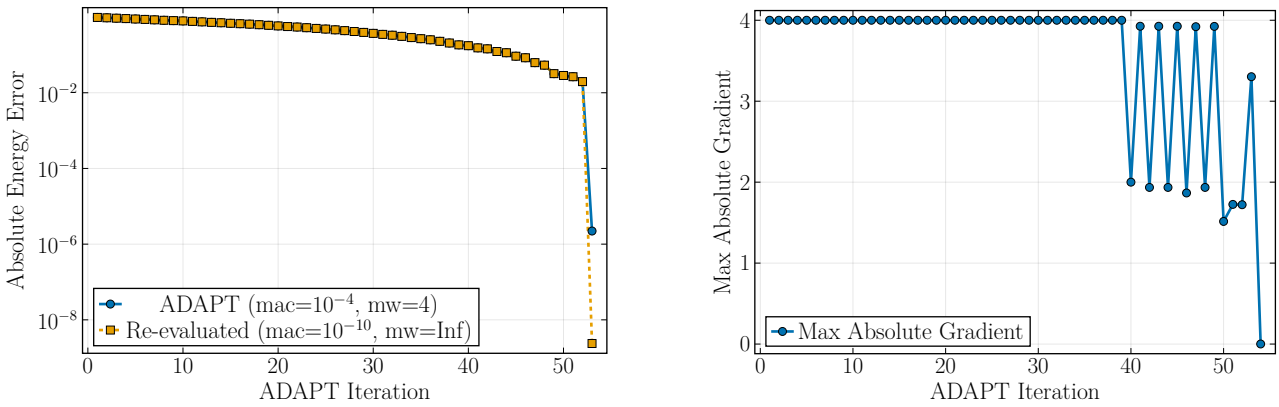
3 Results

3.1 Algorithmic and Software Development

Our main contribution is the design and implementation of a Julia-based, end-to-end ADAPT-VQE framework [29] that enables propagation-driven simulations at larger system sizes with controlled truncations. In particular, we implemented the tiled operator-pool construction in 1D and 2D. Achieving this required building the algorithmic pipeline beyond the textbook ADAPT loop and resolving practical bottlenecks that otherwise dominate runtime and memory.
We engineered the optimization loop around Pauli propagation and implemented a re-evaluation pipeline that recomputes energies (and selected observables) at much stricter truncations or, where feasible, nearly exactly. This re-evaluation is essential in the present setting because truncation breaks strict unitarity and can lead to non-variational energy estimates.
A further part of our contribution is the gradient and commutator infrastructure required to make ADAPT iterations practical. We implemented commutator-based pool-gradient evaluation with truncations for overlap calculations following Equation 2.2.5, and we compared this approach to alternative approximate gradient strategies (e.g., autodifferentiation through truncated circuits). In the regimes tested, commutator-based gradients provided more reliable operator choices for energy reduction.
To ensure that the code is usable for systematic studies, we implemented performance diagnostics (timing and memory, including threaded execution where appropriate). We used these benchmarks to select robust default settings for cluster-scale runs. Additional implementation tests (thread scaling, truncation schedules, tile-size comparisons, optimizer/backend comparisons, and symmetry-observable checks) are collected in the Appendix 5. Finally, we designed the codebase around a type-based interface for Hamiltonians and topologies, allowing users to add new models, lattices, and boundary conditions without modifying the ADAPT core code.

3.2 Simulations of Physical Models

$J_1 - J_2$ **model.** We validate the tiled ADAPT implementation on the J_1-J_2 chain with open boundary conditions by demonstrating a converged run at 90 qubits (Figure 3.1). We perform the ADAPT optimization with moderate truncations (mac = 10^{-4}, mw = 4) and re-evaluate the resulting circuit (including all intermediate parameter sets) at much stricter truncation (mac = 10^{-10}, no weight truncation) to obtain a near-exact reference at feasible cost. The close overlap between the original run and the re-evaluation indicates that the learned parameters are representative and not dominated by truncation artefacts. For this instance, we obtain a V-score of $\sim 10^{-9}$ (Appendix 5.10). Beyond energies, we verify physicality via symmetry diagnostics: the magnetizations satisfy $M_x = M_y = M_z \simeq 0$ with maximal deviations of order 10^{-8}, and SU(2) isotropy holds for spin–spin correlators, $\langle S_i^x S_j^x \rangle = \langle S_i^y S_j^y \rangle = \langle S_i^z S_j^z \rangle$, for nearest neighbors, with maximal deviations on the order of machine precision. To achieve such high accuracy, we reran each simulation 5 times and retained the best energy instance. While the other runs had depths of around 150 layers and relative errors of 10^{-2}, the best run converged in 50 layers with a relative error of 10^{-6} (10^{-8} upon re-evaluation).
An explanation for the ease and accuracy of the Pauli propagation ground state preparation is that at the Majumdar–Ghosh point of the antiferromagnetic J_1–J_2 chain, which we examined, the ground space is exactly spanned by the two nearest-neighbor dimer coverings. These are product states of singlets on alternating bonds up to the expected twofold degeneracy (for periodic chains) associated with translation symmetry breaking. Concretely, for open boundaries, one may write $|\Psi_{\mathrm{MG}}\rangle = \bigotimes_k |s\rangle_{(2k-1,2k)}$ with $|s\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$, so the entanglement present is strictly local (one Bell pair per dimer) [30, 31]. Crucially for Pauli propagation simulations, each singlet $|s\rangle$ is a Bell state that is Clifford-preparable and hence a stabilizer state. Therefore, the Majumdar–Ghosh ground state is a comparably easy problem to study, while allowing us to test the functioning and features of the implemented algorithm. The ground-state structure explains why ADAPT-VQE with a tile size of



(a) Absolute energy error from the exact reference ground state of the J_1-J_2 model on a chain. The original run (which is mostly covered by the re-evaluation) is performed with moderate truncations. In contrast, the re-evaluation of the final circuit, with the intermediate angles at a given ADAPT iteration, is performed nearly exactly.

(b) Matching gradient convergence curve to the energy error curve for the J_1-J_2 model on a chain. The maximum absolute gradients are 4 for the first 40 iterations, then oscillate, and finally fall below the convergence threshold of 10^{-2} after 50 iterations.

Figure 3.1: Energy and gradient convergence for 90 qubits of the J_1-J_2 model on a chain.

two (a pool restricted to two-qubit Pauli strings on nearest-neighbor pairs) efficiently represents this state. A small number of local two-qubit Pauli rotations suffices to synthesize each Bell-pair dimer, so the required circuit depth and parameter count scale linearly with the number of dimers and do not need to build long-range entanglement. Correspondingly, the ADAPT selection rule tends to pick precisely these local entanglers early because they yield the dominant local energy gradients.

Scaling behavior and run-to-run variability. After checking a single run, we can also examine how the relative energy error varies with system size. We examine converged runs up to a system size of 100 qubits on a chain with open boundary conditions (Figure 3.2). Similar diagrams for other examined Hamiltonians are in the Appendix 5.5. We observed that, with the current implemented algorithm, we can prepare the ground state of small systems nearly exactly (to machine precision). In contrast, for larger systems, a relative accuracy of $10^{-1} - 10^{-2}$ is typical. Looking specifically at the $J_1 - J_2$ model, we achieved higher accuracies. The ADAPT run yields a relative error of approximately 10^{-2}, whereas the re-evaluation increases the accuracy by approximately one order of magnitude. This indicates that, although truncation can lead to non-physical or less accurate energy estimates, the variational landscape explored during the ADAPT run remains qualitatively aligned with the ideal objective: the truncated cost function appears to preserve the relevant descent directions and approximate shape of the true energy landscape. Consequently, the parameters and gates learned under truncation are still representative of the target ground state, as confirmed by the improved accuracy obtained upon re-evaluation.
To verify that the found ground state approximately fulfills the symmetry properties of the Hamiltonian, we analysed the magnetization deviation from zero and the nearest-neighbor correlations in section 5.6. These show good overlap with the symmetry properties. Regarding the simulation settings, we use the following options: Analysing different tile sizes for this model has led us to choose a tile size of two (see section 5.4), which corresponds to the building blocks of the HVA ansatz. This also coincides with the fact that working with two-qubit gates is preferred on quantum hardware.
For the optimization backend, we use forward differentiation. We show that this backend achieves accuracies comparable to those of more complex methods while significantly reducing runtime (see section 5.7). Specifically, we can simulate 5 instances of the 100-qubit problem in 17 hours on a single thread on the cluster. Since the peak memory is low, with around 1.7 GB at the set truncations, we can expect to simulate this on a laptop as well, especially if we reduce the number of maximum iterations. An essential factor in the accuracy of the final ground state is the order in which the ADAPT algorithm

chooses the operators. Since for the same maximum gradients, we choose at random which operator to append, the resulting circuits can vary significantly. Therefore, we rerun the simulation multiple times and replace the previous run if the energy is lower. This procedure can yield highly accurate simulations, as the 90-qubit example shows.

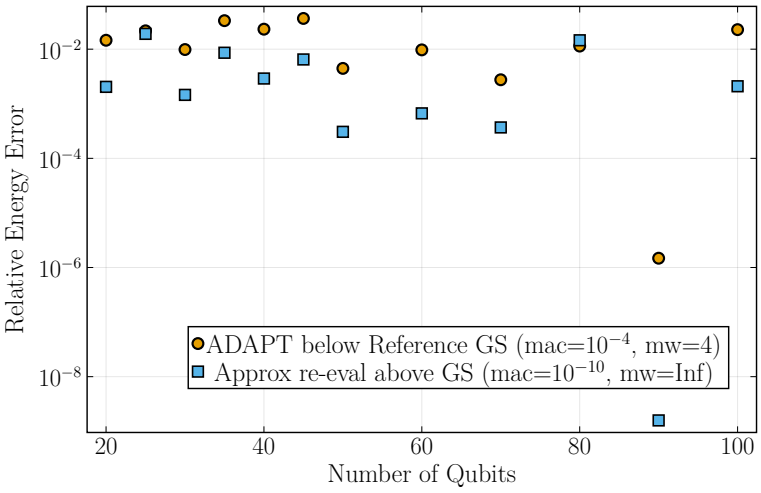


Figure 3.2: Relative error over qubit count for the J_1-J_2 model on a chain with open boundary conditions. Here, all found energies from the truncated run are below the exact reference ground state. Only upon re-evaluation do the energies exceed the ground-state value. The relative error does not grow with system size. The reference ground state is available exactly for an even number of qubits, whereas for an odd number of qubits, we used NQS benchmarks.

The re-evaluations provide the final estimate of the ground-state energy, which we compute as accurately as computationally feasible. Since we re-evaluate at each step, we can verify whether the result is physically consistent, i.e, the exact re-evaluation obeys the variational principle. While exact re-evaluation is only possible for a limited range of system sizes, nearly exact re-evaluation at low truncations can empirically indicate whether we have respected the system's physicality.
Furthermore, the convergence of the gradients provides insight into the plausibility of the results, even before examining energy convergence. When we observe a gradient convergence as in Figure 3.1b, this indicates that the corresponding energy estimate is physically consistent, since we achieved convergence to 0 of the gradients. For non-unitary propagation, we observed that the gradient first plateaus, then oscillates, and, ideally, stagnates. If, instead, it increases again, the truncated results should be treated with caution. Furthermore, if the gradient plateaus, the energy also plateaus in all cases we have observed.
A peculiarity of the ADAPT-VQE algorithm is that, for specific small systems, it converges to the first excited state to machine precision. We suspect this is due to the small absolute error gap, since it does not occur for larger systems ($n > 15$).

Truncation choice and re-evaluation. For each ground-state preparation, we must tune many hyperparameters. Arguably, the most important decision next to the operator pool is the choice of truncation for the Pauli propagation algorithm. In Figure 3.3, the energy convergence of a specific run (50 qubits of the J_1-J_2 model) is shown for multiple truncation settings and the corresponding re-evaluations. We observe that, for a small number of iterations, the almost exact re-evaluation and the truncated run overlap, because the truncations tend to have a more substantial influence the deeper the circuit gets. The highest truncation that we choose is mac $= 10^{-3}$ and mw $= 4$. We need to include at least weight-4 terms, as this is the maximum weight we can achieve with a branching term of a weight-2 Hamiltonian, such as the Hamiltonians we are considering. We observed that this weight choice is the minimum requirement for convergence. For this specific case with mac $= 10^{-3}$, the ADAPT algorithm does not converge but stops at the maximum number of iterations (150), as we can

see in the gradient evolution in Figure 3.4. The gradient oscillates until the final iteration. Decreasing the mac truncation to 10^{-4} leads to convergence according to the finetuned plateau criterion.
The hyperparameters include `min_iters_for_stagnation`, which specifies the iteration at which gradient convergence can occur, since gradients also plateau in the first few iterations of the algorithm (the longer the larger the system size, as seen in Figure 3.1b). We then select the minimum number of consecutive iterations and the fluctuation interval to determine whether the point belongs to the plateau. We also used `reruns=5` here, so each truncation setting is optimised from scratch multiple times. In general, we observe that the lower the truncation level, the faster the convergence and the smaller the difference between the exact run and the re-evaluation. Using fewer truncations yields higher accuracy and, in the analysed case, reduced the total simulation memory footprint and runtime. For the least accurate run (mac = 10^{-3}), we used around 1.3 GB of peak memory and 80 minutes of runtime, whereas for the most accurate run (mac = 10^{-5}), we also used around 1.3 GB of peak memory and 10 minutes of runtime. We also get lower final gradient values with lower truncation settings. Still, lower truncation can sometimes lead to longer convergence times (orange vs. green curve in Figure 3.4).
Another part of the computation that requires truncation is the overlap calculations of the commutators with the propagated state. As discussed in section 2.2, the gradient computation via the commutator is only approximate for the non-unitary computation (Equation 2.2.5).
We compared the effects of no truncation in the gradient computation, truncating the weight to the current run's weight (while keeping the mac untouched), and truncating the mac to the current run's mac (while keeping the weight untouched). We observed that truncating the mac and not truncating it yield similar accuracy, and adding mac truncation speeds up this part of the computation. Truncating at the current run's weight, on the other hand, results in very inaccurate gradient calculations, leading to zero-valued gradients after only a few circuit layers. By taking the conjugate of the commutator, the Pauli weight increases, and we found the major gradient components to be in these higher weight terms. Thus, we opt for the mac truncation of the gradient calculation matching the current run's settings.
Alternatively, one can compute energy gradients by differentiating through the truncated propagation, for example, using automatic differentiation. In this case, the truncation rules become part of the computational graph, and the resulting derivatives are, by construction, consistent with the truncated surrogate. These derivatives aim to minimize the truncated landscape rather than the intended ideal energy landscape. In the ADAPT setting, this mismatch can bias operator selection toward directions favored by truncation artifacts rather than those that most effectively reduce the physical energy. This explains our empirical observation that autodifferentiation-based gradients produced substantially different gradient values and operator choices and were accompanied by convergence issues. In contrast, commutator-based gradients yielded better operator selections, leading to reliable energy convergence.
We also investigated a looped training schedule in section 5.3, where we initialize ADAPT at relatively aggressive truncation settings, and tighten the truncations once the gradients converge. The goal is to reduce the total cost compared to running the complete optimisation at low truncation throughout. In practice, this strategy did not provide a clear advantage in our setting: although the final accuracy after refinement can match that of runs started directly at the lower truncations, the initial high-truncation stage often requires more ADAPT iterations to reach the plateau criterion and therefore produces deeper circuits, which in turn increases the cost of the subsequent low-truncation refinement. Consequently, we did not rely on truncation refinement schedules during optimisation.

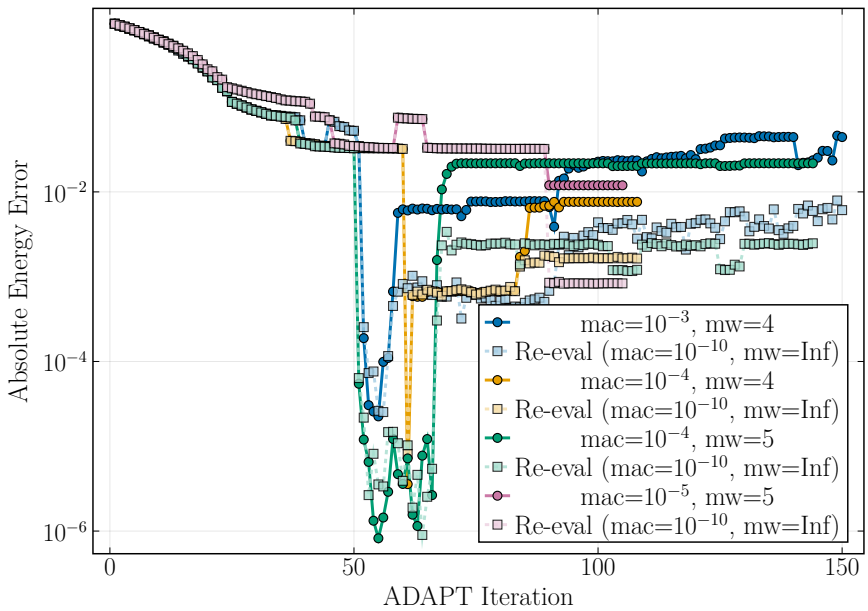


Figure 3.3: Absolute error from ground state energy for 50 qubits with re-evaluation for the J_1-J_2 model on a chain. The original ADAPT and the corresponding re-evaluation are shown in dark and light shades, respectively. At a small iteration count, the truncated run and the re-evaluation overlap. We observe higher re-evaluation accuracy for the lowest truncation (pink) and faster convergence with 100 iterations compared to 150 iterations for the highest truncations (blue).

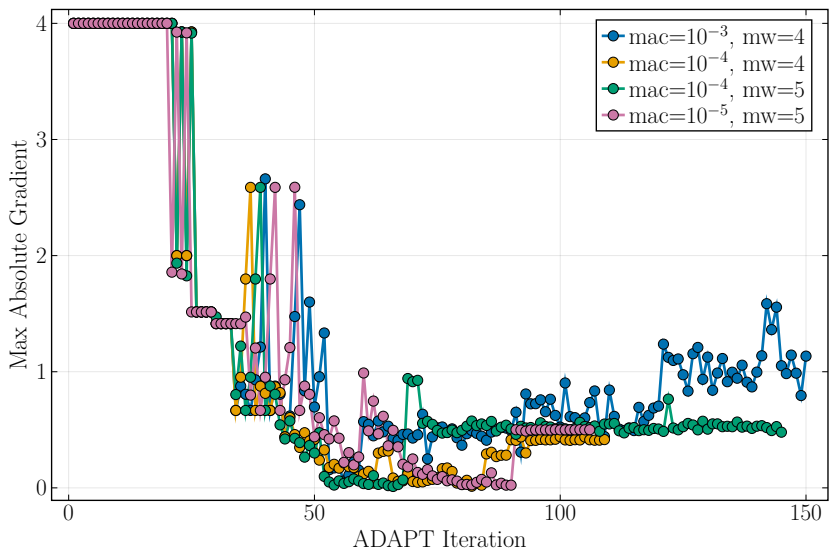


Figure 3.4: Gradient convergence comparison for 50 qubits for the J_1-J_2 model on a chain. Energy convergence is closely linked to gradient convergence. We can see that once the gradient stalls, the energy converges as well.

Heisenberg model. Apart from the $J_1 - J_2$ model, we show the results for the Heisenberg model at $J = 1$ in Figure 3.5. For this system, we do not obtain cluster runs below the ground-state energy (compared to previous J_1-J_2 simulations), and re-evaluations also tend to increase the energy rather than decrease it. The relative accuracies of the re-evaluations are lower than those of the previous model but decrease only slightly with system size.

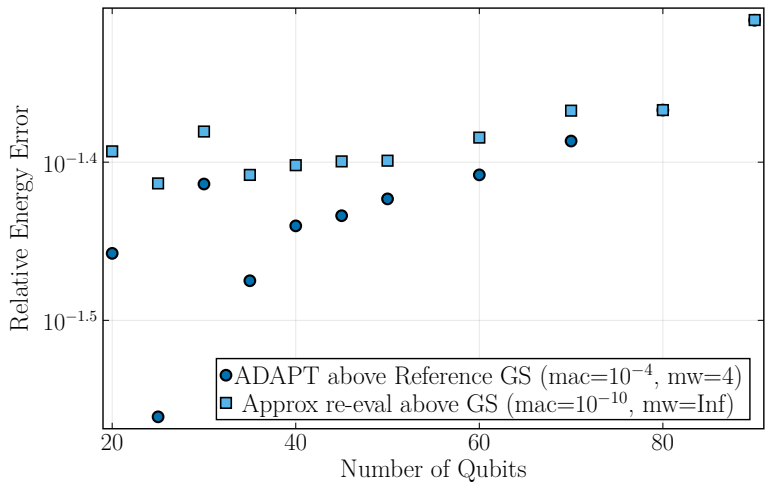


Figure 3.5: Accuracy comparison for the Heisenberg model. The reference energy is computed with DMRG (Density Matrix Renormalization Group). The cluster runs are all above the reference energy, and re-evaluating further increases the energy. This model also demonstrates the scalability in 1D.

Two-dimensional extensions. In addition to simulations on a chain, we extended the tiling approach to two dimensions to simulate the models of interest on a square lattice with open boundary conditions. Periodic boundary conditions correspond to a larger version of the open-boundary case, since the tiles must wrap on all sides until one step before they overlap. We therefore start by studying the open-boundary case. One example is the TFIM in Figure 3.6 on a 4×4 lattice, which converges with a relative error of order 10^{-1} in the re-evaluation after around 40 iterations. Scaling to larger systems than 4×4 with the tiling ansatz yields lower accuracies at the same truncations, so the accuracy is not nearly constant with system size, as observed in the 1D case. To scale up, we will need to run deeper, more accurate simulations. From the tiling paper [23], we see that much deeper circuits are required to achieve higher accuracy in two dimensions. For a relative energy error of 10^{-4} on a 3×3 lattice of the XXZ Hamiltonian, they ran 200 iterations on quantum hardware.

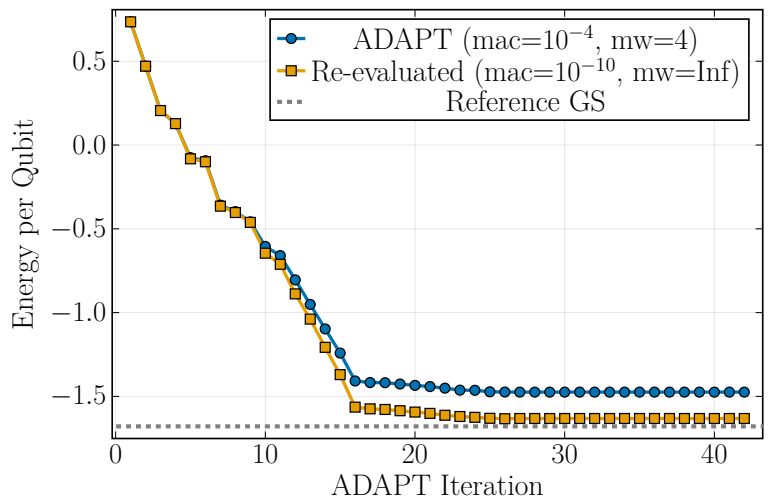


Figure 3.6: Energy convergence comparison for 16 Qubits for the TFIM model on a 2D square lattice (4×4) with ED reference. The truncated run and the re-evaluated run show larger deviations than in the 1D cases.

4 Conclusion

Summary of results. In this work, we developed an algorithmic framework for studying ground-state preparation for spin systems using the Pauli propagation method within the ADAPT-VQE framework. Our implementation demonstrates scalability in 1D, performing approximate ground-state preparation for 1D chain systems with up to 100 qubits. Even highly truncated optimisation runs (around mw = 4, mac = 10^{-4}) agree largely with exact re-evaluations at stricter truncation settings, indicating that the truncated learning captures the dominant contributions to the objective function. Throughout, we relied on numerically stable gradient evaluation and validated the optimisation procedure by re-evaluating the final circuits at each iteration to identify truncation-based artefacts. We validated the accuracy of our results by examining key observables and confirming that the prepared states approximately respect the expected symmetry properties, without explicitly imposing those properties in the pool. These findings highlight the potential and scalability of Pauli propagation to enhance the performance of variational quantum algorithms for simulating complex quantum systems.

Future research directions. Regarding further improvements to the current implementation, a significant challenge remains in improving the accuracy of ground-state preparations, particularly at low truncation orders. The code, for example, enables starting the optimization from an initial circuit. This is made possible by the code's extensible architecture, which allows users to define systems and settings externally while using all our algorithmic implementations. Therefore, we can test physically motivated initializations, such as DQA-initialized HVA layers, which could reduce the initial energy and allow ADAPT to focus on refinement. Since we observed that DQA initialisation can fail under truncated propagation using purely HVA, this needs to be investigated further to design initialisations that are robust under the chosen truncation scheme.
More broadly, an improved training setup would enforce the relevant symmetries during optimisation, either by restricting the operator pool to symmetry-preserving generators or by adding penalty terms that stabilise the target sector.
Additionally, extending ground-state preparation capabilities to simulate a broader range of two-dimensional topologies and quantum-chemistry Hamiltonians, including the Hubbard Hamiltonian and impurity models, is an important research direction. Since standard fermion-to-qubit mappings for two-dimensional fermionic systems scale unfavorably, the goal is to use the Majorana propagation framework to encode fermionic systems directly. To ensure a broad applicability of our code, it will use a shared propagation base for both operator propagation methods, facilitating testing across a wide range of systems and improving maintainability.
In addition to analysing a wider range of systems, a key objective is to bridge the gap between classical simulations and quantum hardware implementations. This hybrid approach involves preparing states classically using Pauli propagation and subsequently executing them on quantum hardware. A central aspect of effectively realising this transfer in the NISQ regime is the careful selection of the operator pool, based, for example, on the hardware topology, and the selection rules within the classical variational algorithm. The goal is to promote the generation of shallow, hardware-executable circuits—i.e., sequences of physically motivated Pauli rotations with low depth and favorable connectivity. This strategy aims to ensure that the resulting warm-start states can be prepared within current coherence times while still capturing the essential correlations of the target ground state.
Furthermore, to address the limitations imposed by circuit depth on quantum hardware, specifically regarding fermionic systems, future research could investigate the application of techniques such as Natural Orbitalization, as introduced in [32]. It is an iterative basis-rotation procedure that uses the measured one-particle reduced density matrix of the current variational state to rotate the fermionic modes toward the (approximate) natural-orbital basis, in which the target ground state can often be prepared with shallower circuits due to a more compact representation.
Another promising avenue for future exploration is the incorporation of variational Density Matrix Backflow (vDBF) techniques. The vDBF algorithm can be seen as an extension of the ADAPT

algorithm, in which only the most recently added layers are optimised, and the operator pool is dynamic. By introducing backflow transformations into the variational ansatz, it may be possible to capture correlation effects more effectively, potentially improving the accuracy of the ground-state energy estimate [22].

Bibliography

[1] Sam McArdle et al. "Quantum computational chemistry". In: *Reviews of Modern Physics* 92.1 (Mar. 2020). ISSN: 1539-0756. DOI: 10.1103/revmodphys.92.015003. URL: http://dx.doi.org/10.1103/RevModPhys.92.015003.

[2] Jules Tilly et al. "The Variational Quantum Eigensolver: A review of methods and best practices". In: *Physics Reports*. The Variational Quantum Eigensolver: a review of methods and best practices 986 (Nov. 2022), pp. 1–128. ISSN: 0370-1573. DOI: 10.1016/j.physrep.2022.08.003. URL: https://www.sciencedirect.com/science/article/pii/S0370157322003118.

[3] John Preskill. "Quantum Computing in the NISQ era and beyond". en-GB. In: *Quantum* 2 (Aug. 2018). Publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften, p. 79. DOI: 10.22331/q-2018-08-06-79. URL: https://quantum-journal.org/papers/q-2018-08-06-79/.

[4] Hayata Morisaki et al. "Classical variational optimization of PREPARE circuit for quantum phase estimation of quantum chemistry Hamiltonians". In: *Physical Review Research* 6.4 (Nov. 2024). arXiv:2308.13770 [quant-ph], p. 043186. ISSN: 2643-1564. DOI: 10.1103/PhysRevResearch.6.043186. URL: http://arxiv.org/abs/2308.13770.

[5] Daniel J. Egger, Jakub Mareček, and Stefan Woerner. "Warm-starting quantum optimization". en-GB. In: *Quantum* 5 (June 2021). Publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften, p. 479. DOI: 10.22331/q-2021-06-17-479. URL: https://quantum-journal.org/papers/q-2021-06-17-479/.

[6] Dian Wu et al. "Variational Benchmarks for Quantum Many-Body Problems". In: *Science* 386.6719 (Oct. 2024). arXiv:2302.04919 [quant-ph], pp. 296–301. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.adg9774. URL: http://arxiv.org/abs/2302.04919.

[7] Román Orús. "A practical introduction to tensor networks: Matrix product states and projected entangled pair states". In: *Annals of Physics* 349 (Oct. 2014), pp. 117–158. ISSN: 0003-4916. DOI: 10.1016/j.aop.2014.06.013. URL: https://www.sciencedirect.com/science/article/pii/S0003491614001596.

[8] F. Verstraete, V. Murg, and J.I. Cirac. "Matrix product states, projected entangled pair states, and variational renormalization group methods for quantum spin systems". In: *Advances in Physics* 57.2 (Mar. 2008). Publisher: Taylor & Francis, pp. 143–224. ISSN: 0001-8732. DOI: 10.1080/14789940801912366. URL: https://doi.org/10.1080/14789940801912366.

[9] Y.-Y. Shi, L.-M. Duan, and G. Vidal. "Classical simulation of quantum many-body systems with a tree tensor network". In: *Physical Review A* 74.2 (Aug. 2006). Publisher: American Physical Society, p. 022320. DOI: 10.1103/PhysRevA.74.022320. URL: https://link.aps.org/doi/10.1103/PhysRevA.74.022320.

[10] F. Verstraete and J. I. Cirac. *Renormalization algorithms for Quantum-Many Body Systems in two and higher dimensions*. arXiv:cond-mat/0407066. July 2004. DOI: 10.48550/arXiv.cond-mat/0407066. URL: http://arxiv.org/abs/cond-mat/0407066.

[11] Jacob Biamonte and Ville Bergholm. *Tensor Networks in a Nutshell*. arXiv:1708.00006 [quant-ph]. July 2017. DOI: 10.48550/arXiv.1708.00006. URL: http://arxiv.org/abs/1708.00006.

[12] Giuseppe Carleo and Matthias Troyer. "Solving the quantum many-body problem with artificial neural networks". In: *Science* 355.6325 (Feb. 2017). Publisher: American Association for the Advancement of Science, pp. 602–606. DOI: 10.1126/science.aag2302. URL: https://www.science.org/doi/10.1126/science.aag2302.

[13] Patrick Rall et al. "Simulation of Qubit Quantum Circuits via Pauli Propagation". In: *Physical Review A* 99.6 (June 2019). arXiv:1901.09070 [quant-ph], p. 062337. ISSN: 2469-9926, 2469-9934. DOI: 10.1103/PhysRevA.99.062337.

[14] Manuel S. Rudolph et al. "Classical surrogate simulation of quantum systems with LOWESA". en. In: arXiv:2308.09109 (Aug. 2023). arXiv:2308.09109 [quant-ph]. DOI: 10.48550/arXiv.2308.09109. URL: http://arxiv.org/abs/2308.09109.

[15] Yuguo Shao et al. "Simulating Noisy Variational Quantum Algorithms: A Polynomial Approach". In: *Physical Review Letters* 133.12 (Sept. 2024). Publisher: American Physical Society, p. 120603. DOI: 10.1103/PhysRevLett.133.120603. URL: https://link.aps.org/doi/10.1103/PhysRevLett.133.120603.

[16] Manuel S. Rudolph et al. *Pauli Propagation: A Computational Framework for Simulating Quantum Systems*. en. May 2025. URL: https://arxiv.org/abs/2505.21606v1.

[17] Nicolas Loizeau, J. Clayton Peacock, and Dries Sels. "Quantum many-body simulations with PauliStrings.jl". In: *SciPost Physics Codebases* (Mar. 2025). arXiv:2410.09654 [quant-ph], p. 54. ISSN: 2949-804X. DOI: 10.21468/SciPostPhysCodeb.54. URL: http://arxiv.org/abs/2410.09654.

[18] Aaron Miller et al. *Simulation of Fermionic circuits using Majorana Propagation*. arXiv:2503.18939 [quant-ph]. Dec. 2025. DOI: 10.48550/arXiv.2503.18939. URL: http://arxiv.org/abs/2503.18939.

[19] Matteo D'Anna, Jannes Nys, and Juan Carrasquilla. *Majorana string simulation of nonequilibrium dynamics in two-dimensional lattice fermion systems*. arXiv:2511.02809 [cond-mat]. Nov. 2025. DOI: 10.48550/arXiv.2511.02809. URL: http://arxiv.org/abs/2511.02809.

[20] Harper R. Grimsley et al. "An adaptive variational algorithm for exact molecular simulations on a quantum computer". en. In: *Nature Communications* 10.1 (July 2019). Publisher: Nature Publishing Group, p. 3007. ISSN: 2041-1723. DOI: 10.1038/s41467-019-10988-2. URL: https://www.nature.com/articles/s41467-019-10988-2.

[21] Cheng-Ju Lin, Hrant Gharibyan, and Vincent P. Su. *Utility-Scale Quantum State Preparation: Classical Training using Pauli Path Simulation*. arXiv:2510.02428 [quant-ph]. Oct. 2025. DOI: 10.48550/arXiv.2510.02428. URL: http://arxiv.org/abs/2510.02428.

[22] Chinmay Shrikhande et al. *Rapid ground state energy estimation with a Sparse Pauli Dynamics-enabled Variational Double Bracket Flow*. arXiv:2511.21651 [quant-ph]. Nov. 2025. DOI: 10.48550/arXiv.2511.21651. URL: http://arxiv.org/abs/2511.21651.

[23] John S. Van Dyke et al. "Scaling adaptive quantum simulation algorithms via operator pool tiling". In: *Physical Review Research* 6.1 (Feb. 2024). Publisher: American Physical Society, p. L012030. DOI: 10.1103/PhysRevResearch.6.L012030. URL: https://link.aps.org/doi/10.1103/PhysRevResearch.6.L012030.

[24] Armando Angrisani et al. "Classically Estimating Observables of Noiseless Quantum Circuits". In: *Physical Review Letters* 135.17 (Oct. 2025), p. 170602. DOI: 10.1103/lh6x-7rc3. URL: https://link.aps.org/doi/10.1103/lh6x-7rc3.

[25] Harper R. Grimsley et al. "Adaptive, problem-tailored variational quantum eigensolver mitigates rough parameter landscapes and barren plateaus". en. In: *npj Quantum Information* 9.1 (Mar. 2023). Publisher: Nature Publishing Group, p. 19. ISSN: 2056-6387. DOI: 10.1038/s41534-023-00681-0. URL: https://www.nature.com/articles/s41534-023-00681-0.

[26] Ho Lun Tang. "Qubit-ADAPT-VQE: An Adaptive Algorithm for Constructing Hardware-Efficient Ansätze on a Quantum Processor". In: *PRX Quantum* 2.2 (2021). DOI: 10.1103/PRXQuantum.2.020310.

[27] Alberto Peruzzo et al. "A variational eigenvalue solver on a photonic quantum processor". en. In: *Nature Communications* 5.1 (July 2014), p. 4213. ISSN: 2041-1723. DOI: 10.1038/ncomms5213. URL: https://www.nature.com/articles/ncomms5213.

[28] V. O. Shkolnikov et al. "Avoiding symmetry roadblocks and minimizing the measurement overhead of adaptive variational quantum eigensolvers". en-GB. In: *Quantum* 7 (June 2023). Publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften, p. 1040. DOI: 10.22331/q-2023-06-12-1040. URL: https://quantum-journal.org/papers/q-2023-06-12-1040/.

[29] Julia Gerecke. *PP-GSP*. GitHub repository. Accessed: 2026-01-21. URL: https://github.com/JuliaGerecke/PP-GSP.

[30] Chanchal K. Majumdar and Dipan K. Ghosh. "On Next-Nearest-Neighbor Interaction in Linear Chain. I". In: *J. Math. Phys.* 10.8 (1969), p. 1388. DOI: 10.1063/1.1664978.

[31] Brijesh Kumar. "Quantum spin models with exact dimer ground states". In: *Physical Review B* 66.2 (June 2002). arXiv:cond-mat/0205072, p. 024406. ISSN: 0163-1829, 1095-3795. DOI: 10.1103/PhysRevB.66.024406. URL: http://arxiv.org/abs/cond-mat/0205072.

[32] P. Besserve and T. Ayral. "Unraveling correlated material properties with noisy quantum computers: Natural orbitalized variational quantum eigensolving of extended impurity models within a slave-boson approach". In: *Physical Review B* 105.11 (Mar. 2022). Publisher: American Physical Society, p. 115108. DOI: 10.1103/PhysRevB.105.115108. URL: https://link.aps.org/doi/10.1103/PhysRevB.105.115108.

[33] Matteo d'Anna and Manuel Rudolph. *MajoranaPropagation.jl*. GitHub repository. A Julia library for Majorana propagation simulation of fermionic quantum circuits and quantum systems. URL: https://github.com/SparqleSim/MajoranaPropagation.jl.

[34] Steven G. Johnson. *The NLopt nonlinear-optimization package*. https://github.com/stevengj/nlopt. 2007.

[35] J. Revels, M. Lubin, and T. Papamarkou. "Forward-Mode Automatic Differentiation in Julia". In: *arXiv:1607.07892 [cs.MS]* (2016). URL: https://arxiv.org/abs/1607.07892.

[36] Tadashi Kadowaki and Hidetoshi Nishimori. "Quantum annealing in the transverse Ising model". In: *Physical Review E* 58.5 (Nov. 1998), pp. 5355–5363. DOI: 10.1103/PhysRevE.58.5355. URL: https://link.aps.org/doi/10.1103/PhysRevE.58.5355.

[37] Glen Bigan Mbeng, Rosario Fazio, and Giuseppe Santoro. *Quantum Annealing: a journey through Digitalization, Control, and hybrid Quantum Variational schemes*. arXiv:1906.08948 [quant-ph]. Dec. 2019. DOI: 10.48550/arXiv.1906.08948. URL: http://arxiv.org/abs/1906.08948.

[38] Glen Bigan Mbeng. "Optimal working point in digitized quantum annealing". In: *Physical Review B* 100.22 (2019). DOI: 10.1103/PhysRevB.100.224201.

[39] Zong-Liang Li and Shi-Xin Zhang. *The Dual Role of Low-Weight Pauli Propagation: A Flawed Simulator but a Powerful Initializer for Variational Quantum Algorithms*. arXiv:2508.06358 [quant-ph]. Jan. 2026. DOI: 10.48550/arXiv.2508.06358. URL: http://arxiv.org/abs/2508.06358.

5 Appendix

5.1 Threading of the Propagation

The propagation-based loss evaluation decomposes naturally at the level of Hamiltonian terms. For an objective of the form

$$E(\underline{\theta}) = \langle \psi(\underline{\theta}) | \hat{H} | \psi(\underline{\theta}) \rangle = \sum_j c_j \langle \hat{P}_j \rangle_{\underline{\theta}},$$

each contribution $\langle \hat{P}_j \rangle_{\underline{\theta}}$ can be obtained by propagating the corresponding Pauli string P_j independently through the circuit and calculating the overlap with the reference state. This formulation enables straightforward parallelization across Hamiltonian terms. The optimal threading strategy on high-performance computers depends on the wall-clock time (i.e., the real-world duration rather than CPU time), peak memory usage, and scheduling overhead. For each Hamiltonian structure, topology, truncation thresholds, and circuit depth, we must perform a threading analysis to optimize resource usage. In the current code base, we provide two practical modes.

Unthreaded (single-task) propagation. In the full Hamiltonian propagation (single-task) scheme, the loss function propagates the Hamiltonian as a single Pauli-sum object through the circuit, applying truncation after each gate to minimize per-term overhead. It is typically the most memory-demanding option because intermediate operator growth is aggregated across all terms before truncation. We therefore opt for this mode when memory is ample (for low circuit depths and small systems), and the truncation is sufficiently aggressive to keep the number of Pauli terms under control. This is how we performed the time and memory analysis of section 5.2, and we refer to it as the unthreaded version.

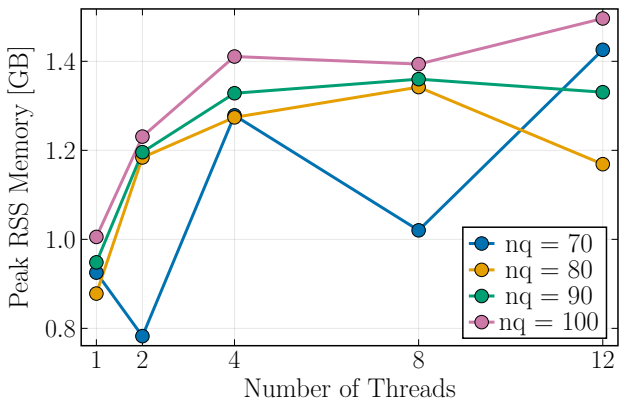
Threaded (block-wise) propagation. Alternatively, we can thread the propagation and optimization by evolving blocks of Hamiltonian terms independently. This can reduce wall-clock time outside the regime where overheads dominate. We implement this via the `ThreadsX` library and set the number of Julia threads at job launch. Increasing the thread count reduces wall-clock time until it reaches a sweet spot, beyond which performance can degrade due to scheduling overhead and load imbalance on the threads. A key limitation of this block-wise strategy is that, in deeper circuits, many blocks tend to generate highly similar intermediate Pauli terms. Since these identical terms are never merged across threads during propagation, multiple workers may end up carrying (and repeatedly updating) essentially the same Pauli-sum content. As a result, parallel efficiency can deteriorate with circuit depth: the deeper the circuit, the more duplicate Pauli strings there are. Therefore, more redundant work is performed across threads, which can limit speedups and reduce the expected memory savings per propagation. In terms of resources, this mode trades time for memory: each worker maintains its own intermediate operator representations, so peak memory increases with the number of concurrently active tasks and the typical intermediate Pauli-sum size.

Numerical non-equivalence under truncation. A practical subtlety is that the two implemented modes are, in general, not numerically equivalent for the same input truncation parameters. In full-Hamiltonian propagation, identical Pauli strings generated by different Hamiltonian terms can merge before truncation, adding their coefficients and potentially retaining contributions that would fall below threshold if treated in isolation. In contrast, term-wise propagation applies truncation independently to each term and then sums only the final scalar contributions, so there is no intermediate merging across terms. We expect larger discrepancies between the two cases in regimes with strong branching and aggressive truncation. Propagating the full Hamiltonian tends to yield larger intermediate Pauli coefficients because the same Pauli string can receive contributions from many terms and is merged before coefficient truncation. Therefore, for a fixed `min_abs_coeff` threshold, full-Hamiltonian propagation is typically more accurate than term-wise propagation, and there is, in general, no uniquely equivalent lower per-term threshold that reproduces the globally merged truncation behavior. Note

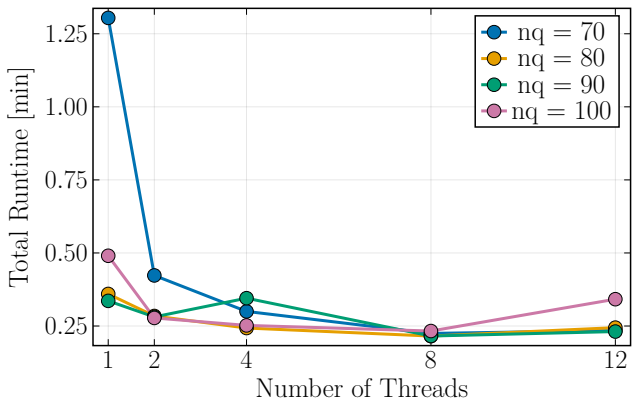
that not all truncation types are affected by threading. For example, maximum-weight truncation depends only on the Pauli string's weight, so it commutes with splitting into blocks and subsequently adding components, up to floating-point effects.

Sequential term-wise propagation (single thread). The most memory-conservative strategy one can use, in principle, is sequential term-wise propagation on a single thread. While this is slower in wall-clock time, it avoids replicated intermediate data structures and reduces peak memory usage to essentially that of a single term propagation (plus accumulation buffers). We can use this approach for both extensive systems and low-truncation cases, where multi-threaded peak memory usage would otherwise be prohibitive. We do not currently provide a dedicated implementation, but it serves as a useful conceptual baseline for the minimum achievable peak memory.

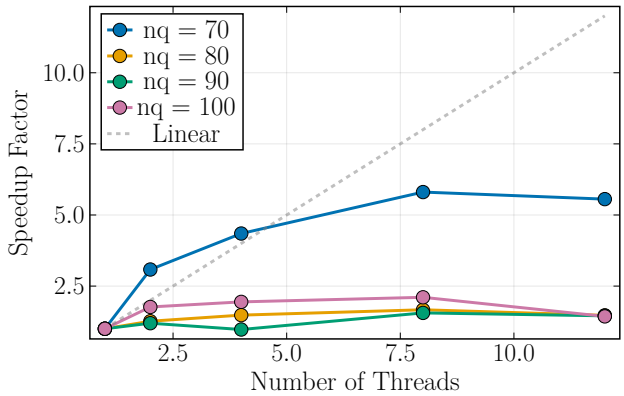
Cluster usage and tuning. On a cluster node, the user empirically selects the thread count for each model and truncation regime. A practical procedure is to benchmark a representative iteration (with fixed circuit depth and truncation settings) across several thread counts and to record both wall time and peak RSS, i.e., the maximum amount of RAM a process has occupied at any single point in time during its entire execution. Typically, one observes an initial speedup region followed by diminishing returns. The optimal configuration is system-dependent: Hamiltonians with many similarly sized terms benefit more from threading than those with a few dominant terms, which is influenced by the underlying topology.
In Figure 5.1, we show an exemplary analysis of the thread count, here specifically for the J_1-J_2 Hamiltonian. To enable a fair comparison, we fix the maximum depth to 50 layers across all system sizes, which is satisfied for all data points. The one-thread data points correspond to using the full loss function (not activating the `ThreadsX` package). For multiple threads, we use the individual loss function for each Pauli string, and the `ThreadsX` package maps them onto the selected number of threads. We observe that peak RSS memory increases with the number of threads and stagnates from 4 threads onwards (Figure 5.1a), but is in a reasonable regime. The total runtime is minimized at 8 threads for all system sizes (Figure 5.1b). The speedup factor depends on the system size; for example, the 70-qubit case with 4 threads exhibits linear speedup (Figure 5.1c). The reason 2 threads can exceed the theoretical linear speedup is that we are not evaluating the same loss functions. For the threaded version, there is no merging, so we drop more Pauli terms, which can speed up the computation for the same input truncation coefficients. A possible reason why 70 qubits require more memory up to a fixed depth of 50 layers is that they may converge more quickly than larger systems, thereby requiring more memory for the final optimization steps. Due to initial short runtimes for the other system sizes, there is no significant speedup for 80-100 qubits.



(a) Peak RSS memory over the number of threads.



(b) Total runtime over the thread count.



(c) Resulting parallel speedup from threading.

Figure 5.1: Threading analysis for the J_1-J_2 model on a chain. One thread corresponds to full operator propagation without thread setup, incurring significant overhead.

5.2 Single Run Timing and Memory Analysis

Unthreaded version. The relevant computational resources for Pauli simulations are runtime and peak memory usage. We compare a non-threaded run and examine how timing usage breaks down across runs, as shown in Figure 5.2. This run has a relative energy error of 10^{-2} as shown in Figure 3.2, and a depth of 150 layers, which is the maximum number of iterations set. While certain runs for large systems achieved convergence at lower depths, we are interested in the typical time and memory usage for a deep circuit. We divide the code into three main parts. The commutators of each Pauli string (trial generator of a Pauli rotation) and the Hamiltonian are calculated only once per ADAPT call. Since this computation is exact, we track the time individually. Next, for each ADAPT iteration, the expectation value of the commutator is calculated for each trial operator. Importantly, we do not perform the computation 2.2.7 exactly because it becomes infeasible as the circuit depth grows; instead, we truncate the mac to the current runs mac, while leaving the mw = Inf. This procedure significantly reduces gradient computation times, making their contribution small relative to the total time. We observed that the algorithm's accuracy is not affected by this truncation choice; ADAPT still chooses the same operators in the cases we examined. Finally, the L-BFGS optimizer's runtime is tracked, with a maximum of 300 iterations using the forward-differentiation backend. The optimizer is the primary contributor and the first aspect to consider when reducing computational time. We observe that the optimization times are close to zero up to 80 iterations, after which they increase linearly using the forward differentiation backend (Appendix 5.7). Regarding memory usage and overall iterations, the peak RSS memory for this run was 1.7 GB, making it, in principle, runnable on a laptop. The peak RSS memory remained stable across system sizes and the circuit depths we examined, indicating further scalability of our current implementation. We also examined the total heap-allocated memory over iterations and observed that, as expected, the total runtime is dominated by the total memory allocations.

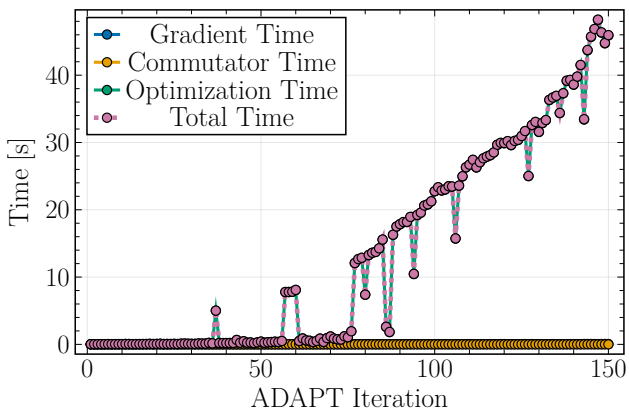


Figure 5.2: Time per ADAPT iteration of 80 qubits for the J_1-J_2 model on a chain. We track three main components of the time contribution. The gradients of all operators in the pool are computed at every ADAPT iteration to select the maximal value. The commutator computations for the operator pool are performed once per ADAPT call (covering the gradient times). The optimization time is the time required by the L-BFGS optimizer using the forward difference method.

5.3 Loop Implementation

Another consideration for ground-state preparation is the choice of truncations at specific points in the optimization. The idea is to explore whether we can benefit from performing the first run of the optimization with a higher truncation to reduce the computational cost, then switching to lower truncations once the previous sequence has converged to increase the accuracy of the solution. A main trade-off is the final depth achieved with this method. If the first truncation is chosen too high

and does not converge (so it is cut off only after the maximum number of iterations), this can be more computationally expensive than initialising at lower truncations and achieving a lower circuit depth. In Figure 5.3, we take a run which begins at mw = 5 and mac = 10^{-5}, and compare this to a run which is initialised at higher truncations and is then refined to the mentioned truncations once the previous values have converged. We observe that refining the truncations yields similar accuracy, albeit at the expense of increased circuit depth. The convergence criteria for a subrun were chosen to match the overall convergence criteria, i.e., observing a plateau in the gradient, since this integrates seamlessly into the algorithm. For a more in-depth view, we compared the accuracy across qubit sizes of both methods, their respective runtimes, and circuit depths (Figure 5.4). While leading to similar accuracies (Figure 5.5), the looped version runs longer in most cases, which is closely linked to the circuit depth. We can therefore deduce that enabling lower-truncation simulations via threading should be prioritized over refining truncations during runs.

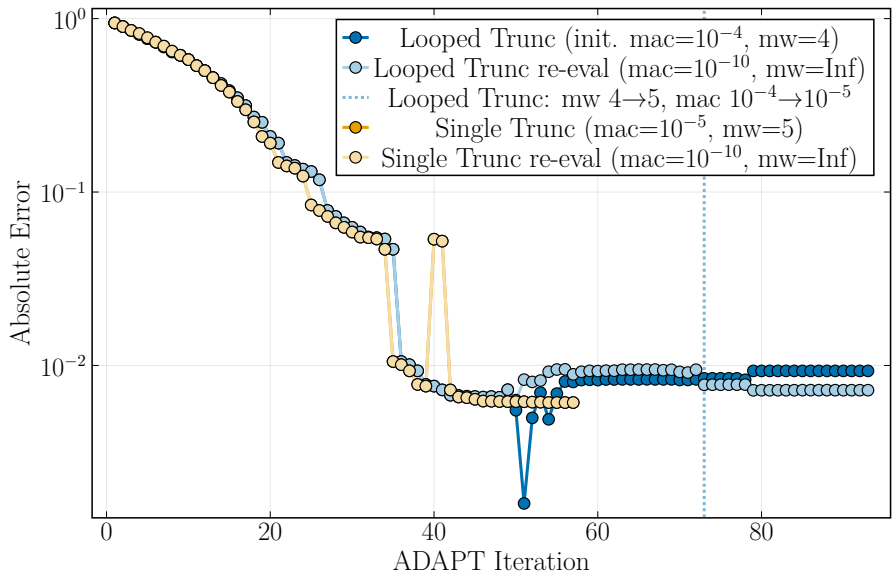
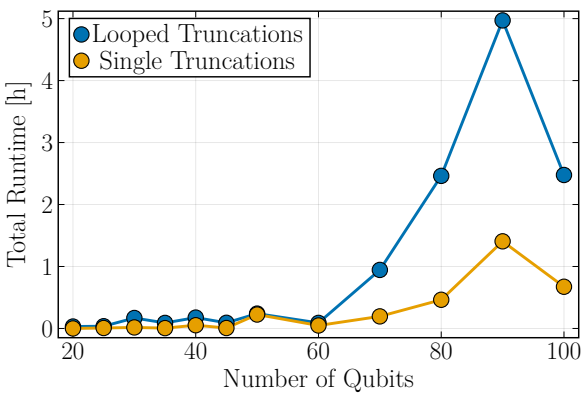
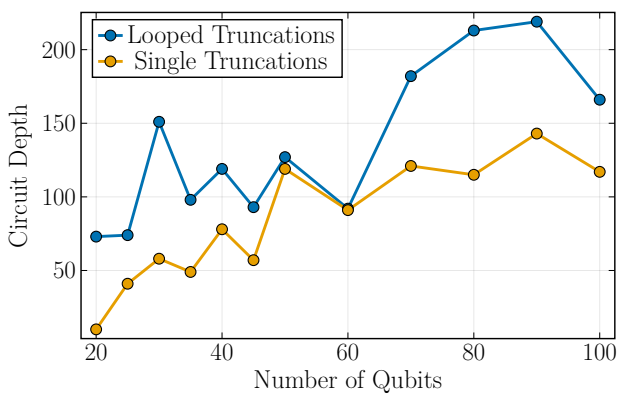


Figure 5.3: Absolute energy error convergence for 45 qubits comparing refined truncations and immediate use of lower truncations. We used an NQS reference benchmark since the exact solution is only available for an even number of qubits. For the regimes tested, immediately using higher accuracy simulations is of benefit.



(a) Time per ADAPT iteration of 45 qubits for the J_1-J_2 model on a chain.



(b) Circuit depth over system size for the J_1-J_2 model on a chain.

Figure 5.4: Total time and circuit depth analysis for unthreaded 45 qubits of the J_1-J_2 model on a chain.

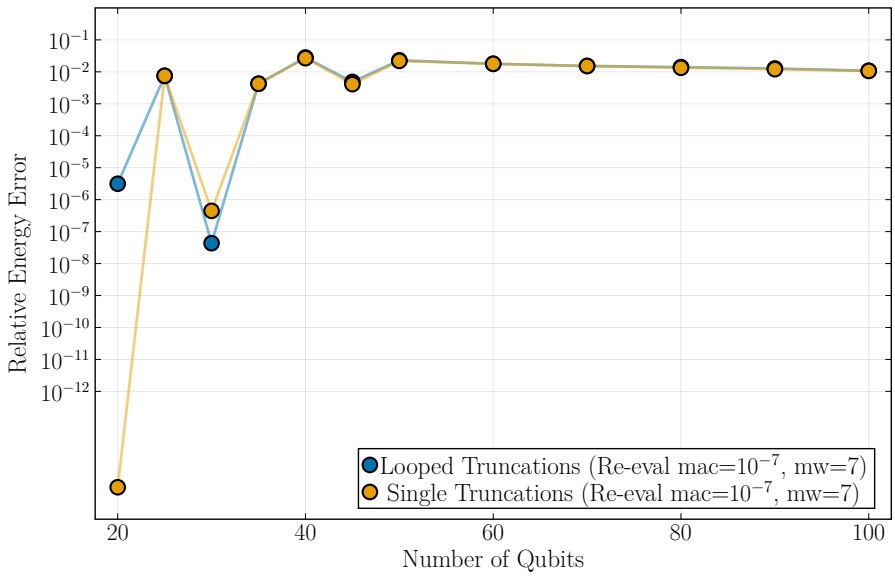


Figure 5.5: Accuracy comparison for refined truncations and immediate use of lower truncations. We do not use reruns here. The looped and non-looped versions achieve similar accuracies across all qubit sizes.

5.4 Tiling Comparison

A further choice when using the tiling approach is the size of the tiles, or the mw of the ansatz operators. For a selected tile size, we run the optimization with the full operator pool 15 times up to a maximal depth of 10 layers. Since we select the appended gate at random for degenerate gradients, multiple runs are necessary to capture all relevant operators. We verified that these hyperparameter choices lead to convergence to the ground state of the smaller system. Figure 5.6 shows the relative energy error over the number of qubits for different tile sizes. We observe that the differences in accuracy are not systematically correlated with tile size, as the curves show no clear preference. An intuitive reason to choose a tile size of two is that this is the maximum weight of terms in the Hamiltonian. Two-qubit operators also form the building blocks of the Hamiltonian Variational Ansatz, which performs well with Pauli propagation [21] at high circuit depths. Furthermore, if we want to work with a prepared ground state for computations on hardware, two-qubit gates are preferred. The original tiling paper uses a tile size of three, but it does not provide intuition for this choice. Choosing a non-ideal tile size can affect the achieved accuracy, as larger tile sizes may exhibit large gradients even if they do not represent the ground state well. Since our analysis shows that the accuracy of the size-2 tiles in one dimension is as good as the other methods, while running in a fraction of the runtime required by larger tile sizes for some data points, as shown in Figure 5.7, this is the best choice for the J_1-J_2 model. Similar analysis can be carried out for other Hamiltonians of interest.

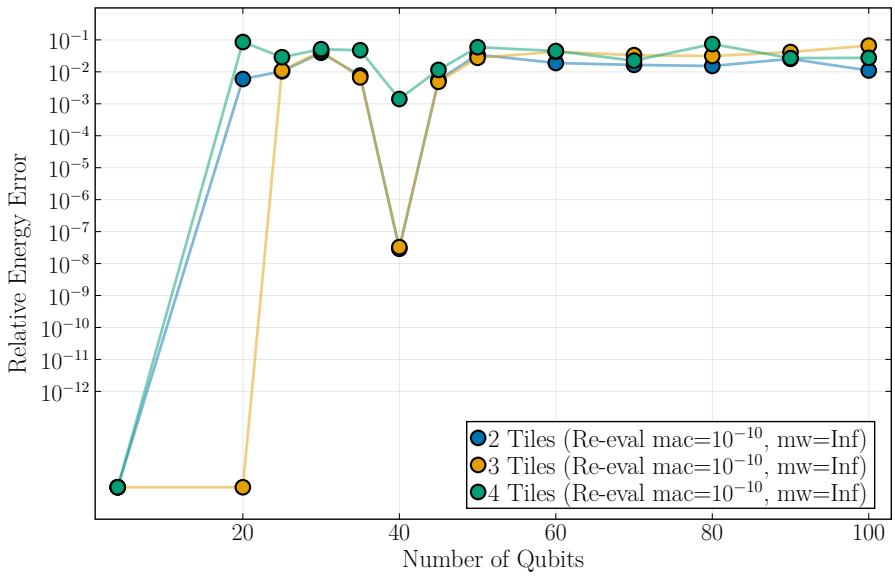


Figure 5.6: Tile size accuracy comparison for J_1-J_2 on a chain. Upon re-evaluation, we achieve similar accuracies across all tile sizes (except for small systems).

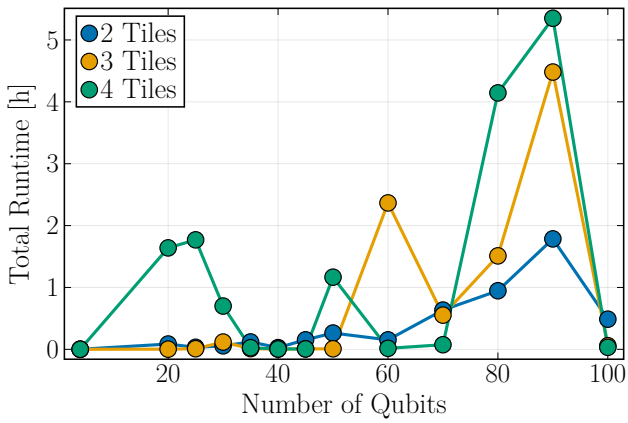


Figure 5.7: Tile size timing comparison for J_1-J_2 on a chain. The overall timing increases with the tile size.

5.5 Spin and Fermionic Hamiltonians

TFIM. The main results focused on the J_1-J_2 and Heisenberg models. We also demonstrate the applicability of our ansatz conception to the TFIM Hamiltonian introduced in section 2.3. We present results for the relative accuracy of the energy as a function of qubit count for the TFIM. In Figure 5.8, the relative energy errors from the ground state energy for the TFIM are shown. We calculated the reference energies via the available exact solution for open boundary conditions in 1D and reran the PP simulations 5 times. We further ran NQS simulations as an additional point of comparison. We see that the accuracies of PP (both before and after re-evaluation) are between 10^{-1} and 10^{-2}, with a slight decrease in accuracy as the system size increases. This is expected since we ran all simulations with the same initial truncations. As we can see, even if the truncated run leads below the ground-state energy (as in the 20-qubit case), the approximate re-evaluation is above at all times. This indicates that we are learning the correct ground state and have not violated a fundamental symmetry of the system, since the variational principle holds, as expected.

t-V and Hubbard model. Regarding the t-V model at half-filling, we used a Jordan-Wigner transformation to map the spinless fermions onto qubits/Pauli strings. When running the ADAPT-VQE as above and performing the exact re-evaluations, we observed some system sizes that yielded values

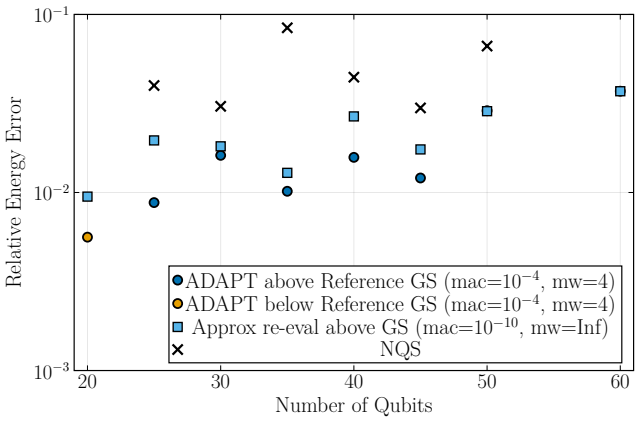


Figure 5.8: ADAPT-VQE accuracy for TFIM on a chain. Our ground state preparation method achieves higher accuracy than the NQS comparison data upon re-evaluation. The scaling with system size is also favorable for same truncation settings.

below the reference ground-state energy from DMRG. The naive ADAPT ansatz does not conserve particle number. We tracked how the particle number changes with ADAPT iterations, slowly drifting away from the initial number when not explicitly constrained. The easiest way to preserve this symmetry in the ansatz is to restrict the operator pool to the relevant symmetries by implementing a manual pool with tile-sized elements, which can then be scaled rather than relying on automatic selection via small ADAPT runs. The operators that preserve particle number in the Jordan-Wigner transformation are rotations of Pauli sums, which can be decomposed into Pauli rotations since the generating operator summands commute. Thus, working with the Jordan-Wigner mapping would require adding multiple operators at each layer and binding the corresponding optimization parameters. In general, the Jordan-Wigner approach is only a temporary solution for fermionic simulations. This is because, in 2-dimensional simulations, the non-locality of the fermion-to-qubit mapping prevents efficient scaling. Instead, we will work directly in the Majorana Propagation (MP) framework, using the unified PP and MP implementation [33].

For the Hubbard model, we also performed simulations using the tiles approach. A main observation is that while we can perform accurate simulations of the Fermi-Hubbard model using a Jordan-Wigner fermion-to-qubit mapping and the full operator pool for small systems, the tile-based approach fails because the operators proposed by the ADAPT algorithm are local in the Pauli basis, but no longer after the Jordan-Wigner transformation. Due to the fermion-to-qubit mapping, two spin sites correspond to one fermionic site. Therefore, we considered a tiles approach with a stride (shift of the tile) of two instead of one, since this respects the assigned number of sites per fermion. Nevertheless, this method, combined with the Jordan-Wigner method, was not sufficiently expressive to simulate using the tiles approach. Thus, to work with fermionic models more complex than the t-V model, we must work directly within the MP framework, since this allows us to select relevant operators that model the ground state.

5.6 Observables for Symmetry Verification

To verify ground-state preparation, it is essential to examine the symmetry properties of the analysed Hamiltonian and to assess how well the found ground state satisfies these constraints, since observing the energy is only an indirect measure of the ground state and requires additional verification. Namely, depending on the model, the Hamiltonian $\hat{H}$ commutes with one or several conserved quantities $\{\hat{Q}_\alpha\}$ (e.g. particle number, total magnetisation, parity, lattice symmetries), i.e. $[\hat{H}, \hat{Q}_\alpha] = 0$. This implies a block structure of $\hat{H}$ into symmetry sectors labelled by the eigenvalues q_α of $\hat{Q}_\alpha$. If the variational state leaves the target sector during optimisation, the reported energy may not describe the intended system: it may correspond to a different symmetry sector, and in approximate simulation settings, spurious symmetry-breaking can occur, potentially yielding artificially low energies. For this reason,

symmetries should ideally be enforced during the optimisation and verified a posteriori via dedicated observables.

Imposing symmetries by operator pool restriction or penalty terms. A robust way to enforce the Hamiltonian's symmetries in the ansatz is to restrict the variational manifold to a fixed symmetry sector. We choose an initial state $|\psi_0\rangle$ that is an eigenstate of each $\hat{Q}_\alpha$ with the desired eigenvalues q_α, and an ansatz built from unitaries that preserve these eigenvalues. An ADAPT circuit

$$|\psi(\underline{\theta})\rangle = \prod_{k=1}^{L} e^{-i\theta_k P_k} |\psi_0\rangle, \tag{5.6.1}$$

remains in the target subspace if every generator P_k commutes with all conserved quantities,

$$[P_k, \hat{Q}_\alpha] = 0 \qquad \forall\, k, \alpha, \tag{5.6.2}$$

since then $e^{-i\theta_k P_k}$ preserves the symmetry sector and $|\psi(\underline{\theta})\rangle$ cannot leak out by construction. In the code, we can implement a manually selected pool of symmetry-preserving operators. We can then tile this manual pool in the same way as we did in the previous tiling approach. If a symmetry-adapted pool is inconvenient (e.g. due to hardware constraints or a limited gate set), one can enforce the desired sector by adding a penalty term to the cost function. For target eigenvalues q_α, a standard choice is

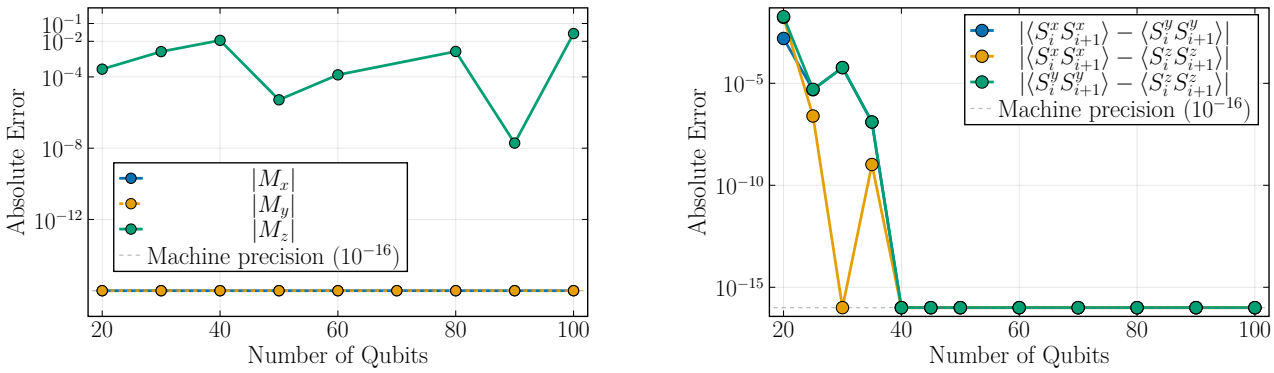
$$E_{\underline{\lambda}}(\underline{\theta}) = \langle\psi(\underline{\theta})|\, \hat{H}\, |\psi(\underline{\theta})\rangle + \sum_\alpha \lambda_\alpha \langle\psi(\underline{\theta})|\, \left(\hat{Q}_\alpha - q_\alpha\right)^2 |\psi(\underline{\theta})\rangle, \tag{5.6.3}$$

where $\lambda_\alpha > 0$ is a hyperparameter chosen to suppress states outside the desired sector. These approaches provide a principled way to guarantee that the optimisation remains in the intended symmetry sector. In the simulations reported in this thesis, however, we do not impose explicit symmetry constraints in the ansatz for the spin Hamiltonians (consistent with [23]), and instead validate the optimised states a posteriori by measuring symmetry-sensitive observables and checking their deviations from the expected values.

Symmetry analysis for the J_1-J_2 model For certain models (Heisenberg, J_1-J_2), we can, for example, expect the magnetization in all directions to be 0 ($M_x = M_y = M_z = 0$), where

$$M_x = \frac{1}{N}\sum_i^N \hat{\sigma}_i^x. \tag{5.6.4}$$

The absolute error of the magnetizations (deviation from 0) is shown in Figure 5.9a. Both M_y and M_z pass this check to machine precision, whereas M_x exhibits varying accuracy, but remains in a reasonable regime for scaled systems. Furthermore, we can perform an isotropy check. Since the J_1-J_2 and the Heisenberg model have an SU(2) symmetry, spin-spin correlations should be isotropic, i.e $\langle S_i^x S_j^x\rangle = \langle S_i^y S_j^y\rangle = \langle S_i^z S_j^z\rangle$. For J_1-J_2, i and j can be either nearest neighbors or next-nearest neighbors. These should be measured at the centre of the chain to avoid boundary effects in the open-boundary case. From this quantity, we can construct correlators such as $|\langle S_i^x S_j^x\rangle - \langle S_i^y S_j^y\rangle|$ which should all give results close to 0. In Figure 5.9b, the absolute error of the nearest-neighbour correlation check is shown for varying system size. For small systems, we still observe boundary effects, whereas for larger systems, all checks are satisfied to machine precision.
We can further look at observables that need comparison data from other methods, such as the phase diagram of the TFIM in Figure 5.10. We observe that the Z-magnetization check is satisfied, and the NQS and PP results overlap. Regarding the X-magnetization, NQS and PP agree on most data points, with non-negligible deviations near the critical point.
In general, we can observe that even without imposing symmetries, the resulting ground state approximately fulfills the symmetry properties examined. For the spin Hamiltonians, this simplified approach yielded satisfactory results, whereas for the fermionic t-V model under the Jordan-Wigner



(a) Absolute error of magnetization for all three magnetizations over the system size for the J_1-J_2 model on a chain. The M_x and M_y satisfy this check, whereas M_z exhibits deviations.

(b) Measure of the isotropy violations of the J_1-J_2 model over the number of qubits. The deviation from zero is small for all measured correlators.

Figure 5.9: Symmetry-based observables for the J_1-J_2 model.

transformation, disregarding symmetry properties led to unphysical results. For future simulations with fermionic Hamiltonians, constraining the pool is crucial for accuracy.

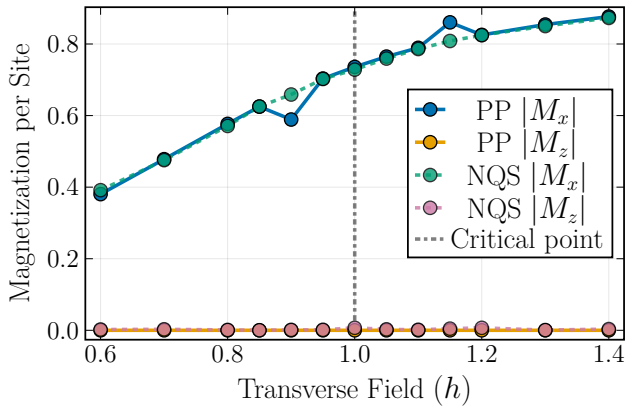


Figure 5.10: Magnetization for the TFIM on a 10-qubit chain with open boundary conditions. The absolute M_z expectation value of the PP simulation and the NQS simulation overlap. For the absolute M_x values, we observe relatively large deviations in the PP simulation at two points, whereas the other points closely follow the NQS benchmarks.

5.7 Optimization Algorithm

We optimize the variational circuit parameters using gradient-based local optimization, where the dominant computational cost arises from repeated evaluations of the energy and its gradient with respect to the circuit parameters, as shown in section 5.2. Since we compute the objective by classical operator propagation with controlled truncations, the resulting loss function is deterministic. We want an optimizer that is robust to moderate noise and non-smoothness, and an automatic differentiation (AD) backend with predictable memory usage.

L-BFGS optimizer. For the VQE inner loop, we employ the limited-memory BFGS (L-BFGS) algorithm as implemented in `NLopt.jl` [34]. L-BFGS is a quasi-Newton method: rather than forming the Hessian of the loss explicitly, it builds an approximation to the inverse Hessian from local curvature information encoded in the most recent parameter and gradient differences $(s_k, y_k) = (\underline{\theta}_{k+1} - \underline{\theta}_k, \nabla E_{k+1} - \nabla E_k)$, where $\underline{\theta}_k \in \mathbb{R}^n$ denotes the n-dimensional parameter vector at iteration k, $E(\underline{\theta})$ is the energy loss function and $\nabla E_k \equiv \nabla E(\underline{\theta}_k)$ is its gradient evaluated at $\underline{\theta}_k$. Each iteration

computes a descent direction $p_k \approx -H_k \nabla E(\underline{\theta}_k)$, where H_k denotes the current L-BFGS approximation to the inverse Hessian, using only a fixed window of m stored curvature pairs, and then performs a line search to choose a step length $\alpha_k > 0$ for the update $\underline{\theta}_{k+1} = \underline{\theta}_k + \alpha_k p_k$. This algorithm typically yields substantially faster convergence than first-order methods, while maintaining a memory footprint of $\mathcal{O}(mn)$, counting the storage for $(s_i, y_i) \in \mathbb{R}^n$ over the most recent m iterations. In practice, using the L-BFGS optimizer yields low peak RSS memory values (up to 2 GB) across all system sizes tested, as described in section 5.2.

Use in variational ground-state preparation. In our setting, the objective and its gradient are deterministic because energies are evaluated by classical propagation. Still, the coefficient truncation can introduce irregularities upon re-calculation when small coefficients cross truncation thresholds. To facilitate convergence, the optimizer is warm-started across ADAPT iterations: previously optimized angles are reused, and only the newly appended parameter is chosen randomly, so each inner-loop solve starts close to an optimum and benefits from L-BFGS' rapid local convergence. In practice, `NLopt.jl` requires supplying both the loss and an in-place gradient; we compute the gradient with one of the backends introduced below, cap the number of optimizer iterations to bound the runtime, and re-evaluate the final circuit to obtain our final result.

Forward difference. In the Julia implementation, we primarily use `ForwardDiff.jl` [35] for the gradient evaluation. The key reason is that forward-mode AD has minimal infrastructure overhead. It does not require constructing or storing a reverse-pass computational graph (called "tape"), so its memory footprint is therefore more stable across iterations. `ForwardDiff` computes derivatives by augmenting floating-point numbers with infinitesimal perturbations, i.e., by propagating dual numbers

$$x \mapsto x + \epsilon \dot{x},$$

where $\dot{x}$ encodes directional derivative information. When the loss function is evaluated on dual-number inputs, all arithmetic and elementary operations are overloaded to carry both the primal value and derivative components. We obtain the gradient by evaluating the loss on a set of dual-number directions, implemented efficiently via `ForwardDiff`'s chunking mechanism. For a parameter vector $\underline{\theta} \in \mathbb{R}^n$, forward-mode computes $\nabla_{\underline{\theta}} E$ in chunks of size c, requiring approximately $\lceil n/c \rceil$ loss evaluations while keeping only local derivative data in memory. We can use the chunk size to balance runtime and memory. Larger chunks reduce the number of loss evaluations but increase per-evaluation memory due to wider dual-number payloads, whereas smaller chunks have the opposite trade-off.
In terms of runtime, this scaling is comparable to finite-difference gradients, requiring a number of loss evaluations linear in the number of parameters. In our setting, where the cost of a single loss evaluation typically increases with circuit depth, the overall gradient time often grows roughly quadratically with the parameter count ($\mathcal{O}(n^2)$). In practice, `ForwardDiff` is typically slightly faster than finite differences and more numerically stable since it avoids subtractive cancellation from differencing nearly equal energies.

Reverse difference. `ReverseDiff.jl` can be attractive when the number of parameters is very large, and the loss evaluation is smooth and static, because reverse-mode AD can compute gradients in a cost comparable to a small constant multiple of one primal evaluation. In other words, for a parameter vector $\underline{\theta} \in \mathbb{R}^n$ (with n variational parameters), the gradient evaluation cost is, in principle, $\mathcal{O}(1)$ evaluations up to a large method-dependent constant factor. However, this advantage relies on the ability to record and reuse a tape that accurately represents the computational graph. In our application, the propagation algorithm includes branching and truncation decisions that vary with the parameter vector $\underline{\theta}$ and the iteratively increasing circuit depth, leading the tape to become stale. In practice, this backend can require frequent re-recording of the tape, thereby eroding much of the theoretical advantage.
A second practical issue is that reverse-mode performance depends on bounding the internal branching complexity of the propagation (e.g., the number of path splits induced by Pauli branching), which is controlled by the `max_freq` truncation in PP. With truncation heuristics, this "effective branching

factor" varies during optimization, and as ADAPT-VQE grows the circuit, it varies further. Thus, a dynamic truncation rule that adapts to the circuit depth would be required to use this truncation effectively.

Mooncake.jl. We also support `Mooncake.jl` (through `DifferentiationInterface.jl`) as an alternative reverse-mode backend. Conceptually, Mooncake differentiates the program itself: it generates derivatives for the exact executed computation, so the gradient corresponds to precisely the implemented (and therefore truncated) loss evaluation, rather than to an idealised, untruncated objective. This can be advantageous in our setting because it avoids differentiation of operations that are later discarded by truncation, and it makes the resulting gradients consistent with the actually optimised cost function. Similar to the previous reverse-mode systems, Mooncake may offer speedups when the loss evaluation is stable and the parameter dimension is large. Still, it likewise benefits from reusing a prepared gradient object tape and may require periodic re-preparation when the circuit structure or internal allocations change. In practice, we find that Mooncake can have a higher memory footprint than the other AD backends considered here, which limits its usefulness for the largest instances.

Backend comparison. In Figure 5.11, we compare the accuracy of the three implemented methods. They show similar results across system sizes. Since we did not rerun, the results may vary upon re-execution. A primary constraint of the mooncake backend is that the runtime becomes prohibitive due to its memory usage at intermediate system sizes when we replace the tape only once per new ADAPT layer (Figure 5.12). Both `ReverseDiff` and `ForwardDiff` perform similarly both in accuracy and runtime. Since the `max_freq` truncation requires a setting tailored to the current layer of the ansatz, we use forward difference in all our simulations.

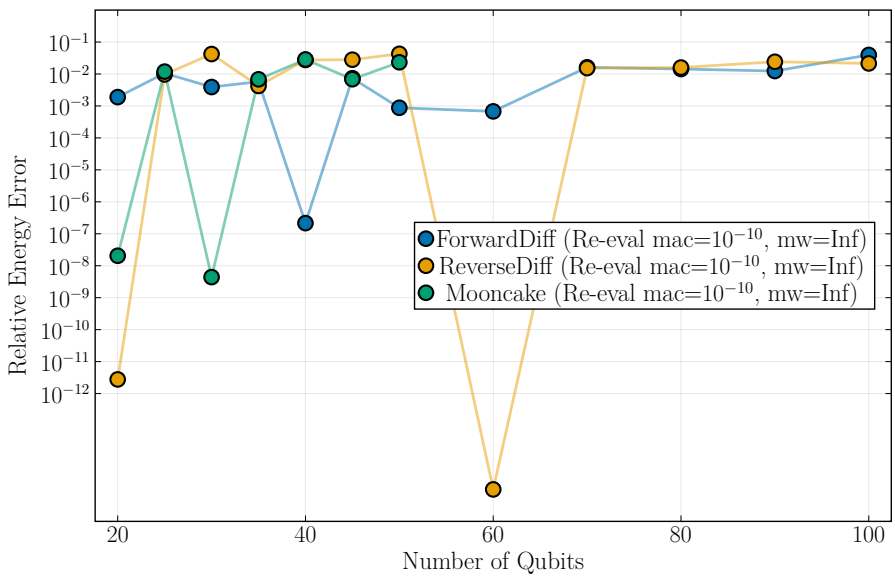


Figure 5.11: Backend accuracy comparison for the J_1-J_2 model on a chain.These results are for a single run, not for multiple runs, and are selected based on the lowest energy. The backend does not seem to be the deciding factor on the relative energy error for the $J_1 - J_2$ model.

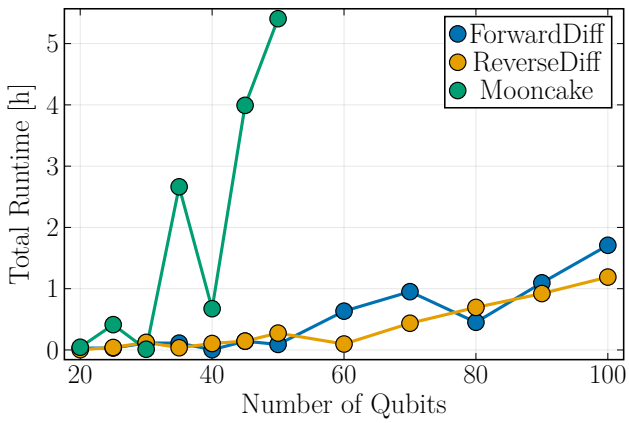


Figure 5.12: Backend timing comparison for J_1-J_2 in 1d. The `Mooncake` backend becomes unfeasible with our settings for system sizes above 50 qubits, while the `ForwardDiff` and the `ReverseDiff` have a similar scaling in time.

5.8 Further Implemented Ground State Preparation Algorithms

In addition to the ADAPT algorithm, which is the focus of this thesis, we tested other ground-state preparation techniques using Pauli propagation. First, we examined the Hamiltonian Variational Ansatz [21] and reproduced the results for the TFIM on a 100-qubit chain. There, we achieved a similar level of accuracy to that reported in the original paper. Both approaches (ADAPT and HVA) have trade-offs: while HVA has a fixed size, ADAPT is an iteratively constructed ansatz. HVA uses $3 \cdot n \cdot l$ parameters, and had $l = 10$ or $l = 20$ layers in their implementation. The ansatz has 3000 parameters for 100 qubits, whereas ADAPT can achieve fewer than 1000 layers. To prepare states for quantum hardware, we need to choose an ansatz that inherently aims for low circuit depth.
Another issue with the HVA is its strong dependence on initialization. We tried both random initialisation and a physically motivated initialisation, the Digitized Quantum Annealing approach [36–38]. We fix the number of layers and initialize the parameters at the adiabatic annealing angles corresponding to this number of steps. We observe that, for this physically inspired angle initialisation, the initial energy is much lower at the beginning of the optimization. Still, the algorithm does not converge to the ground state in our tests for medium-sized systems. At present, it is not clear whether this is due to the optimizer becoming trapped in local minima induced by the greedy DQA initialisation, or whether truncation effects in Pauli propagation distort the optimisation landscape in a way that is particularly detrimental for this initialisation (e.g., by discarding high-weight contributions early that do not reappear under subsequent truncated updates). The latter explanation is supported by the observation that statevector-based training with the same DQA initialisation converges reliably, suggesting that the non-convergence observed here is primarily an artefact of truncation. In comparison, a random initialisation, even starting at higher energies, is more reliable at finding the ground state. The performance of HVA is thus highly dependent on the optimization landscape and the parameter initialization, while ADAPT is more robust.

5.9 Comparison to Previous Propagation-Based Results

There have been several implementations of ground state preparation using operator propagation methods (see Refs. [18, 21, 22, 24, 39]). We briefly compare two of these implementations to our current work.
The work on the Hamiltonian Variational Ansatz reports a truncation of mac $= 10^{-3}$, but does not specify whether this corresponds to weight truncation or another truncation rule. Instead of automatic differentiation, it employs the SPSA-ADAM procedure for gradient estimation, which requires only two function evaluations per gradient and integrates well with GPU implementations. We attempted to apply SPSA–ADAM within our ADAPT-VQE workflow, but did not observe convergence. In practice,

SPSA-ADAM is a noisy optimizer whose performance is highly sensitive to hyperparameter choices: adopting the hyperparameter settings used for HVA did not succeed for ADAPT, so additional tuning would be required to leverage this efficient gradient-estimation approach in our implementation.
Using Majorana propagation, a 28-mode fermionic instance (corresponding to 56 qubits) is simulated with ADAPT-VQE under a modified weight-truncation scheme reaching an absolute error on the order of 10^{-5} mHa. No additional truncations, such as coefficient truncation, are specified. For the 28-mode system, the reported circuits remain below 50 layers in depth. Achieving this level of accuracy at such modest depth suggests that the appended operators provide a highly expressive parametrization of the relevant low-energy manifold. We suspect that leveraging the Hamiltonian's symmetry properties will help us achieve more effective and expressive operator pools for ground-state preparation.

5.10 V-score Computation

The V-score is an intensive, dimensionless combination introduced by [6]. It is a measure of the accuracy of a learned variational state with N degrees of freedom (number of spins for spin systems, number of particles for fermionic systems), an energy variance of $\mathrm{Var}(E)$, and a constant E_∞ which acts as the zero point in energy:

$$\text{V-score} := \frac{N\ \mathrm{Var}(E)}{(E - E_\infty)^2}. \tag{5.10.1}$$

The V-score is invariant under energy shifts. It is a reliable quantity due to its linear relation to the relative energy error for known ground state energies. For methods such as NQS, access to the variance is a key component of the algorithm. For Pauli propagation, we do not have direct access to this quantity and must perform a separate calculation. The most straightforward way to access the variance is by the standard formula: $\mathrm{Var}(E) = \langle H^2 \rangle - \langle H \rangle^2$. The second expectation value of the energy has already been calculated, but should be re-evaluated exactly. We now need to backpropagate the observable H^2 through the optimized circuit and its parameters. This computation (especially when performed exactly) can quickly become infeasible because the number of Pauli strings grows quadratically for most physical Hamiltonians, i.e., the scaling is unfavorable. Thus, an exact computation of the V-score is only feasible for small systems. For larger systems, we cannot compute the V-score exactly and must truncate in both computations. This suggests that a different measure of accuracy can be appropriate for operator propagation.